A PROJECT REPORT

on

MULTI-AGENT AI SYSTEMS FOR HOSPITAL RESOURCE OPTIMIZATION, CLINICAL WORKFLOW AUTOMATION, AND PATIENT CARE COORDINATION

(MedOrchestra — Multi-Agent Hospital Intelligence Platform)

Submitted in partial fulfilment of the requirements for the award of the degree of

_____

Submitted by

Name: _____ Roll No.: _____

Name: _____ Roll No.: _____

Under the guidance of

_____

Department of _____

_____ (Institute)

Academic Year 2026 – 2027

Certificate

This is to certify that the project report entitled **"Multi-Agent AI Systems for Hospital Resource Optimization, Clinical Workflow Automation, and Patient Care Coordination"** is a bonafide record of the work carried out by _____ (Roll No. _____) and _____ (Roll No. _____) under my supervision and guidance, in partial fulfilment of the requirements for the award of the degree of _____ of _____ during the academic year 2026–2027.

The work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma.

_____ _____

Project Guide Head of the Department

External Examiner: _____ Date: _____

Declaration

We hereby declare that the project work entitled **"Multi-Agent AI Systems for Hospital Resource Optimization, Clinical Workflow Automation, and Patient Care Coordination"** submitted to _____ is a record of original work done by us under the guidance of _____. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma. Wherever we have used the work of others, it has been duly acknowledged and referenced.

Place: _____
Date: _____ Signature(s) of the student(s)

Acknowledgement

We express our sincere gratitude to our project guide, _____, for the constant guidance, encouragement and valuable feedback throughout this project. We are thankful to the Head of the Department, _____, and to all faculty members of the Department of _____ for providing the facilities and support needed to complete this work.

We also thank the doctors and nursing staff whose descriptions of everyday hospital workflows — admissions, triage, bed turnover, theatre lists and ward rounds — helped us understand the real problems this system tries to address. Finally, we thank our families and friends for their continuous support and motivation.

Abstract

Hospitals coordinate beds, intensive-care units, operating theatres, doctors, nurses, diagnostics and pharmacy through disconnected systems, telephone calls and spreadsheets. The result is poor resource utilisation, long waiting times, scheduling conflicts and delayed escalation of deteriorating patients. This project designs and implements **MedOrchestra**, a multi-agent artificial-intelligence platform for hospital resource optimisation, clinical workflow automation and patient-care coordination.

Instead of a single AI model, nine specialised agents — Bed Management, Appointment & Queue, Staff Scheduling, Diagnostic Coordination, Pharmacy, Patient Care Coordinator, Emergency Support, Resource Optimization and Notification — are coordinated by a central orchestrator. A natural-language request is classified into one or more of 12 operational intents, converted into a dependency-ordered plan, and executed by agents that share a blackboard and exchange FIPA-ACL-style messages. Decisions are powered by established techniques: Google OR-Tools CP-SAT constraint optimisation for bed allocation, staff rostering and theatre scheduling; the National Early Warning Score 2 (NEWS2) for triage and ward deterioration; M/M/c queueing theory for waiting-time prediction; Holt-Winters exponential smoothing for demand forecasting; and a Gradient-Boosting model with quantile intervals for length-of-stay prediction.

Every recommendation carries an explainable factor breakdown. Low-risk, reversible actions execute automatically, while safety-critical actions (admissions, emergency assignments, AI-suggested investigations, contraindicated prescriptions, discharges, surgeries, rosters) are routed to authorised clinicians for approval, modification or rejection under a configurable autonomy policy, keeping humans in control of clinical decisions.

The system is implemented with FastAPI (Python), SQLAlchemy (SQLite/PostgreSQL), React with TypeScript as an installable progressive web app, and WebSockets for live updates; Claude can optionally be used for language understanding. Evaluation on a synthetic 108-bed hospital shows 89.6% intent accuracy on held-out requests, admission-forecast MAPE of 7.87% (versus 11.43% for a seasonal-naive baseline), a length-of-stay MAE 21% lower than a median baseline, a 7-day roster for 77 staff solved optimally in 2.0 s, and theatre re-optimisation that removed all overlaps and 75 minutes of overtime. All 38 automated tests pass.

Keywords: multi-agent systems, hospital management, resource optimisation, constraint programming, NEWS2, human-in-the-loop, explainable AI, clinical workflow automation.

Table of Contents

Certificate i
Declaration ii
Acknowledgement iii
Abstract iv
Table of Contents v
List of Abbreviations viii
Chapter 1: Introduction 1
1.1 Background 1
1.2 Motivation 1
1.3 Problem Statement 1
1.4 Objectives 2
1.5 Scope of the Project 3
1.6 Organisation of the Report 3
Chapter 2: Literature Survey 4
2.1 Multi-Agent Systems 4
2.2 AI and Optimisation in Hospital Operations 4
2.3 Early Warning and Triage 4
2.4 Queueing Theory in Healthcare 5
2.5 Demand Forecasting 5
2.6 Length-of-Stay Prediction 5
2.7 Large Language Models and Human Oversight 5
2.8 Research Gap 5
Chapter 3: System Analysis and Requirements 7
3.1 Existing System 7
3.2 Proposed System 7
3.3 Feasibility Study 8
3.4 Users and Roles 8
3.5 Functional Requirements 9
3.6 Non-Functional Requirements 9
3.7 Hardware and Software Requirements 10
Chapter 4: System Design 11
4.1 System Architecture 11
4.2 Multi-Agent Orchestration Pipeline 12
4.3 Agent Catalogue 12
4.4 Agent Interaction 13

4.5 Human-in-the-Loop Design 14
4.6 Ward Early-Warning Escalation 16
4.7 Data Flow Diagrams 16
4.8 Use Case Diagram 17
4.9 Database Design 17
4.10 API Design 18
Chapter 5: Implementation 20
5.1 Technology Stack 20
5.2 Project Structure 20
5.3 Orchestrator 20
5.4 Natural-Language Understanding 21
5.5 Specialised Agents 21
5.6 Algorithms 23
5.7 Security, Audit and Real-Time Updates 25
5.8 Frontend and Progressive Web App 25
5.9 Synthetic Data and Live Simulation 25
Chapter 6: Results and Screenshots 27
6.1 Login and Role-Based Access 27
6.2 Command Center 27
6.3 AI Agent Console 29
6.4 Human-in-the-Loop Approvals 33
6.5 Emergency Department 34
6.6 Inpatients and Patient Record 35
6.7 Resources 38
6.8 Clinical Workflows 40
6.9 Analytics, Administration and Patient Portal 42
Chapter 7: Testing and Evaluation 46
7.1 Testing Strategy 46
7.2 Automated Test Results 46
7.3 Scenario Testing 47
7.4 Evaluation of the AI Components 48
7.5 Discussion 49
Chapter 8: Conclusion and Future Scope 51
8.1 Conclusion 51
8.2 Limitations 51
8.3 Future Scope 51
References 52

Appendix A: Installation and Execution 54
Appendix B: Demo Accounts and Walkthrough 55

List of Abbreviations

Table 0.1: Abbreviations used in this report

Abbreviation	Meaning
ACL	Agent Communication Language (FIPA)
ACVPU	Alert, new Confusion, Voice, Pain, Unresponsive
API	Application Programming Interface
CP-SAT	Constraint Programming – Satisfiability solver (Google OR-Tools)
DAG	Directed Acyclic Graph
ED / ER	Emergency Department / Emergency Room
EHR / HIS	Electronic Health Record / Hospital Information System
HDU	High Dependency Unit
HITL	Human-in-the-Loop
ICU	Intensive Care Unit
JWT	JSON Web Token
LLM	Large Language Model
LOS	Length of Stay
MAE / MAPE	Mean Absolute Error / Mean Absolute Percentage Error
NEWS2	National Early Warning Score 2
NLP	Natural Language Processing
OPD	Out-Patient Department
OT	Operation Theatre
PWA	Progressive Web App
RBAC	Role-Based Access Control
TAT	Turnaround Time
TF-IDF	Term Frequency – Inverse Document Frequency

Chapter 1: Introduction

1.1 Background

A hospital is a complex socio-technical system. Patients move between registration, consultation, diagnostics, admission, surgery, intensive care and discharge, while a limited pool of resources — beds, ICU units, operating theatres, ventilators, imaging machines, doctors, nurses and medicines — must be shared among them. In many hospitals these resources and workflows are managed through separate departmental systems, manual communication, spreadsheets and telephone calls. Information about a free ICU bed, an available surgeon or a pending laboratory result often reaches the right person late, or not at all.

Artificial intelligence can help, but a single monolithic model is a poor fit for hospital operations: the sub-problems are very different in nature (triage is a clinical scoring problem, rostering is a combinatorial optimisation problem, appointment booking is a search problem, stock management is an inventory problem), and they interact through shared resources. This project therefore adopts a **multi-agent architecture**, in which specialised AI agents each own one operational domain and a central orchestration layer coordinates them toward a common goal.

1.2 Motivation

- Delays in bed allocation and emergency coordination directly affect patient outcomes; early escalation of deteriorating ward patients saves lives.
- Clinicians spend a significant share of their time on coordination work (finding beds, calling departments, chasing results) instead of patient care.
- Resources such as ICU beds and theatres are expensive; small improvements in utilisation and scheduling produce large operational gains.
- AI systems in healthcare must be explainable, auditable and supervised by qualified professionals — a design requirement rather than an afterthought.

1.3 Problem Statement

Hospital operations involve continuous coordination between patients, doctors, nurses, laboratories, pharmacies, administrative staff and infrastructure. When these activities are handled through disconnected systems or manual communication, the following problems arise:

- **Fragmented information** — patient data, appointments, bed availability, diagnostics, staff schedules and pharmacy information live in different systems.
- **Manual resource allocation** — beds, ICU units, theatres, equipment and staff are allocated by hand, causing inefficiency and conflicts.
- **Long patient waiting times** — consultations, diagnostics and departmental workflows are not coordinated.
- **Communication gaps** — updates between reception, clinicians, laboratory, pharmacy and billing are delayed or missed.
- **Emergency coordination** — high-acuity cases require rapid, simultaneous mobilisation of beds, teams and diagnostics.
- **Administrative workload** — scheduling, notifications, status updates and follow-ups are repetitive.
- **Lack of predictive planning** — historical data is not used to anticipate demand, staffing needs or bottlenecks.

Core problem: there is a need for an intelligent, coordinated system that optimises hospital resources, automates routine workflows and facilitates patient-care coordination through multiple specialised AI agents, while retaining human oversight for critical decisions.

1.4 Objectives

1. Develop a centralised hospital-management platform based on a multi-agent AI architecture.
2. Create specialised agents for resource management, clinical workflow automation and patient-care coordination.
3. Optimise allocation of beds, ICU units, doctors, nurses, equipment, diagnostic facilities and operating theatres.
4. Automate routine workflows such as appointment scheduling, queue management, notifications and follow-ups.
5. Reduce patient waiting time through cross-department coordination and queue prioritisation.
6. Provide intelligent support for emergency prioritisation and resource mobilisation.
7. Analyse historical and real-time data to forecast demand and identify bottlenecks.
8. Provide role-based dashboards for administrators, doctors, nurses and other personnel, and a patient portal.
9. Maintain a human-in-the-loop approach so that critical medical decisions remain with qualified professionals.

10. Provide an extensible architecture that can integrate with existing hospital information systems.

1.5 Scope of the Project

The project delivers a working, end-to-end prototype for a simulated 108-bed multispeciality hospital (320 patients, 86 staff, 48 items of equipment and 180 days of operational history). It covers the twelve modules proposed in the synopsis — authentication and RBAC, patient management, bed and resource management, appointments and queues, staff scheduling, diagnostics, pharmacy, patient-care coordination, emergency support, AI resource optimisation, notifications and analytics — and extends them with ward early-warning monitoring, operating-theatre scheduling, machine-learning length-of-stay prediction, predictive equipment maintenance and a configurable autonomy policy. Integration with real hospital information systems, SMS gateways and IoT devices is represented by interfaces and simulation, and all patient data is synthetic.

1.6 Organisation of the Report

Chapter 2 reviews the literature on multi-agent systems and AI for hospital operations. Chapter 3 analyses the requirements and feasibility. Chapter 4 presents the system design, including architecture, agent interaction, human-in-the-loop design and database design. Chapter 5 describes the implementation and the algorithms used by each agent. Chapter 6 presents the results through screenshots of the working system. Chapter 7 covers testing and quantitative evaluation. Chapter 8 concludes the report and outlines future work.

Chapter 2: Literature Survey

2.1 Multi-Agent Systems

An agent is a computer system situated in an environment that is capable of autonomous action to meet its design objectives; intelligent agents are additionally reactive, proactive and social [1]. A multi-agent system (MAS) consists of several interacting agents, each with incomplete information or capability, whose combined behaviour solves problems beyond any individual agent. MAS research provides two ideas used directly in this project: the **blackboard architecture**, in which independent knowledge sources cooperate by reading and writing a shared data structure under the control of a coordinator [3], and **agent communication languages** such as FIPA-ACL, in which messages carry a performative (inform, request, propose, agree, refuse) that makes the intent of each interaction explicit [2].

2.2 AI and Optimisation in Hospital Operations

Operational research has a long history in healthcare. Operating-room planning and scheduling alone has a large literature covering case-mix planning, surgeon assignment and daily sequencing under uncertainty [7]. Bed management is usually treated as an assignment problem with hard constraints (bed type, isolation, equipment) and soft preferences (ward speciality, distance, workload). Modern constraint-programming solvers such as Google OR-Tools CP-SAT make it practical to solve realistic instances of these problems in seconds [13], which is why constraint optimisation, rather than a learned model, is used for allocation decisions in this project.

2.3 Early Warning and Triage

The National Early Warning Score 2 (NEWS2), published by the Royal College of Physicians in 2017, standardises the assessment of acute-illness severity using six physiological parameters plus supplemental oxygen, and prescribes a graded clinical response [4]. NEWS has been shown to discriminate patients at risk of cardiac arrest, unanticipated ICU admission and death within 24 hours [5]. Emergency departments additionally use five-level triage systems such as the Manchester Triage System, which assigns target times to treatment for each priority band [6]. The Emergency Support Agent and the ward early-warning service implement these published rules transparently rather than replacing them with an opaque model.

2.4 Queueing Theory in Healthcare

Queueing models describe the relationship between arrivals, service capacity and waiting time. The Erlang-C formula for the M/M/c queue, originally derived for telephone exchanges [21], is widely used to size clinics and emergency departments [8], [15]. It shows the strongly non-linear growth of waiting time as utilisation approaches one, which motivates capacity alerts at around 85 % utilisation.

2.5 Demand Forecasting

Hospital demand (admissions, emergency visits, occupancy) exhibits trend and strong weekly seasonality. Holt-Winters triple exponential smoothing models level, trend and seasonality with three smoothing constants [9], [10]. It is interpretable, needs little data and forecasts reliably over short horizons, making it suitable for operational planning.

2.6 Length-of-Stay Prediction

Predicting length of stay supports discharge planning and bed-capacity forecasting. A recent systematic review reports that machine-learning methods — particularly tree ensembles — are among the most frequently used and best-performing approaches, while highlighting the need for well-defined evaluation against baselines [12]. Gradient boosting builds an additive ensemble of shallow trees by functional gradient descent [11]; with quantile loss it also yields prediction intervals, which communicate uncertainty to planners.

2.7 Large Language Models and Human Oversight

Large language models can map free-text requests to structured representations, making systems usable through natural language. However, WHO guidance on AI for health emphasises human autonomy, transparency, accountability and safety [16]. This project therefore uses language models only for understanding and summarisation, constrains their output with a JSON schema, grounds extracted entities against the hospital database, and never lets a model execute safety-critical actions without human approval.

2.8 Research Gap

Table 2.1: Gap analysis

Existing approach	Limitation	How MedOrchestra addresses it
Departmental HIS modules	Siloed data; no cross-department coordination	Unified data model and an orchestrator that spans departments

Existing approach	Limitation	How MedOrchestra addresses it
Stand-alone optimisation tools	Solve one problem (e.g. rostering) in isolation	Specialised agents share a blackboard; the optimizer reasons about beds, staff and waiting patients jointly
Single end-to-end AI model	Opaque, hard to validate, mixes unrelated tasks	Domain agents using transparent, validated methods; factor-level explanations
Fully automated decision systems	Unsafe for clinical decisions; poor accountability	Risk-tiered human-in-the-loop gate, configurable autonomy, full audit trail
Paper-based early warning	Late escalation, no link to bed/ICU capacity	NEWS2 scoring on every observation with automatic ICU-transfer proposals

Chapter 3: System Analysis and Requirements

3.1 Existing System

In a typical hospital the admitting doctor telephones bed managers to find a free bed, nurses record observations on paper charts, the operation-theatre list is maintained on a whiteboard or spreadsheet, OPD queues are managed by token number only, and pharmacy stock is checked manually. Information is duplicated across departments, there is no single view of hospital capacity, escalation depends on individual vigilance, and decisions are rarely recorded with their rationale.

3.2 Proposed System

MedOrchestra replaces this with a single platform in which every patient, resource and workflow is linked. Staff interact through role-specific dashboards or simply describe a need in natural language; the orchestrator plans the workflow, the specialised agents evaluate options with appropriate algorithms, explainable recommendations are presented, low-risk actions are executed immediately and critical actions wait for approval by an authorised person. The key features are:

- Natural-language AI Agent Console with a live visualisation of agents and a step-by-step trace.
- Human-in-the-loop approval queue with approve, reject and approve-with-modification, and three autonomy modes.
- Command centre with live KPIs, ward occupancy, emergency board, bottlenecks and role-specific work lists.
- Emergency department board with live NEWS2 triage preview and automatic multi-agent coordination.
- Inpatient census with bedside observations, NEWS2 trends, observation schedules and automatic ICU-transfer proposals.
- Bed, equipment and operation-theatre management, including CP-SAT theatre optimisation and predictive maintenance.
- OPD appointments with dynamic priority queues and M/M/c wait prediction; staff rostering with CP-SAT.
- Diagnostics and pharmacy workflows with turnaround tracking and drug-safety screening.

- Analytics with demand forecasting, length-of-stay model card and agent performance; audit log; patient portal PWA.

3.3 Feasibility Study

Technical feasibility

The system uses mature open-source technologies (Python, FastAPI, SQLAlchemy, scikit-learn, OR-Tools, React). All algorithms run on commodity hardware: the largest optimisation problem in the prototype (a 7-day roster for 77 staff) solves in about two seconds on a laptop.

Economic feasibility

All core components are free and open source. The optional language-model service is pay-per-use and can be disabled, in which case the local NLP pipeline is used at no cost. Deployment requires a single server or a small container cluster with PostgreSQL.

Operational feasibility

The system supports rather than replaces staff. Adoption can start with low-risk workflows (appointments, bed status, notifications) in supervised mode and gradually extend to optimisation and early warning as trust grows. Role-based dashboards keep each user focused on relevant information.

Data and integration feasibility

The data model mirrors standard HIS entities, and REST APIs allow integration with existing EHR, laboratory and pharmacy systems. The quality of recommendations depends on timely and accurate data, which is why every recommendation shows the facts it is based on.

3.4 Users and Roles

Table 3.1: User roles

Role	Main tasks in the system
Administrator	Monitor operations, approve rosters and redeployments, set autonomy policy, manage users, review audit log, run simulation
Doctor	Ask agents, approve admissions, investigations, prescriptions overrides, surgeries and discharges, review patients
Nurse	Record observations, register emergency cases, update bed status, approve bed placements
Receptionist	Register patients, manage OPD appointments and queues
Lab technician	Progress diagnostic orders and report results
Pharmacist	Verify and dispense prescriptions, manage stock, approve purchase orders

Role	Main tasks in the system
Patient	View own record, reports and medicines; book appointments through the Appointment Agent

3.5 Functional Requirements

Table 3.2: Functional requirements

ID	Requirement
FR-01	Users shall authenticate with username and password; access shall be restricted by role.
FR-02	Staff shall submit operational requests in natural language; the system shall identify intents and entities.
FR-03	The orchestrator shall select agents, build a dependency-ordered plan and execute it with a live trace.
FR-04	The system shall recommend beds, staff and equipment with explainable scores and optimise allocation globally.
FR-05	Safety-critical actions shall require approval by an authorised role; approvers may modify or reject them.
FR-06	Emergency cases shall be triaged with NEWS2 and red-flag detection and coordinated automatically.
FR-07	Bedside observations shall be NEWS2-scored and escalated according to the NEWS2 response policy.
FR-08	The system shall manage OPD appointments, prioritise queues and predict waiting time.
FR-09	The system shall generate optimised staff rosters and propose redeployments.
FR-10	The system shall schedule diagnostic tests on appropriate modalities and track turnaround.
FR-11	The system shall screen prescriptions for allergies, interactions, dose and stock, and plan reorders.
FR-12	The system shall schedule surgeries in operating theatres without overlap and re-optimise the day.
FR-13	The system shall track each patient’s care pathway and produce discharge summaries.
FR-14	The system shall send role-based notifications in real time.
FR-15	The system shall forecast demand, predict length of stay and detect bottlenecks.
FR-16	All user decisions and agent actions shall be recorded in an audit log.
FR-17	Patients shall view their record and book appointments through a portal.
FR-18	Administrators shall manage users and the autonomy policy.

3.6 Non-Functional Requirements

Table 3.3: Non-functional requirements

Category	Requirement
Security	JWT authentication, PBKDF2-SHA256 password hashing, server-side RBAC on every endpoint, audit logging

Category	Requirement
Safety	No safety-critical action without human approval (except under an explicitly configured policy); rejections roll back holds
Performance	Interactive API responses under 1 s; typical multi-agent orchestration completes in a few seconds
Usability	Role-specific dashboards, natural-language input, explanations, global search, responsive/PWA interface
Reliability	Agent failures are isolated; LLM failures fall back to local NLP; allocation transactions are serialised
Maintainability	Modular agents and services, typed API, automated tests
Portability	Runs on Windows/Linux/macOS; SQLite for development, PostgreSQL in Docker for production

3.7 Hardware and Software Requirements

Table 3.4: Hardware and software requirements

Item	Minimum specification
Processor / RAM	Dual-core 2 GHz / 4 GB (8 GB recommended)
Storage	1 GB free disk space
Operating system	Windows 10/11, Linux or macOS
Runtime	Python 3.10+ and Node.js 18+
Database	SQLite (bundled) or PostgreSQL 16
Browser	Any modern browser (Chrome, Edge, Firefox, Safari)
Optional	Docker Desktop; Anthropic API key for Claude-based understanding

Chapter 4: System Design

4.1 System Architecture

The system follows the layered architecture proposed in the synopsis (Figure 4.1). Requests flow from the client layer through the application layer to the AI orchestration layer, which coordinates the specialised agents; agents use shared hospital services and algorithms and persist state in the data layer. Events flow back to all connected clients over a WebSocket bus.

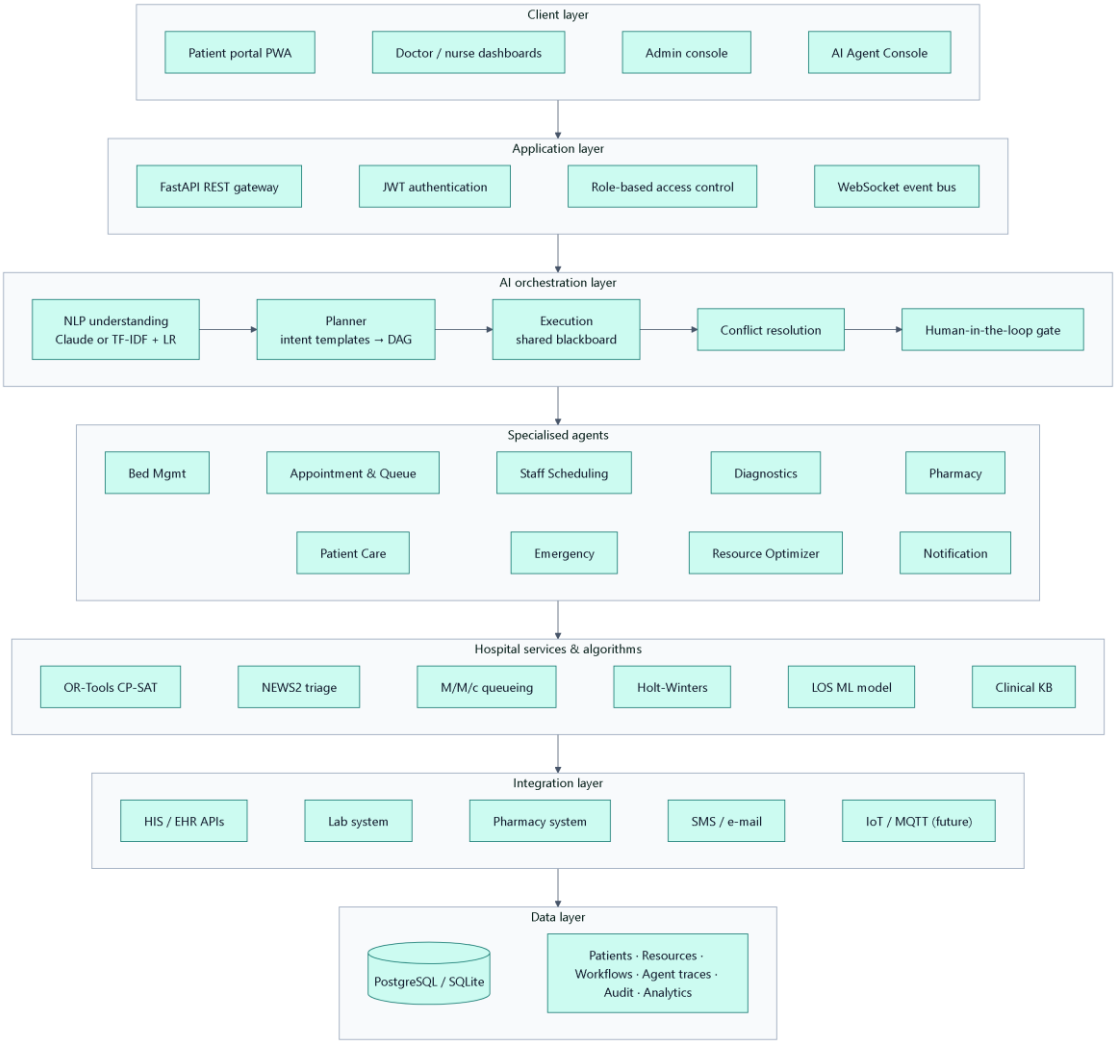


Figure 4.1: Layered system architecture

- **Client layer** — a React/TypeScript single-page application with role-based dashboards, the AI Agent Console and the patient portal, installable as a PWA.
- **Application layer** — FastAPI REST gateway with JWT authentication, role-based access control and a WebSocket event bus.
- **AI orchestration layer** — NLP understanding, planning, blackboard-based execution, conflict resolution and the human-in-the-loop gate.
- **Specialised agents** — nine domain agents that evaluate options and propose actions.

- **Services & algorithms** — optimisation, triage, queueing, forecasting, machine learning, clinical knowledge base, audit and notifications.
- **Integration & data layers** — adapters for external systems and a relational database.

4.2 Multi-Agent Orchestration Pipeline

Every request passes through the ten-step pipeline defined in the synopsis (Figure 4.2, Table 4.1).



Figure 4.2: Orchestration pipeline (synopsis steps i–x)

Table 4.1: Pipeline steps and their implementation

Step	Synopsis stage	Implementation
i	Request intake	AgentRequest record; sources: console, emergency desk, patient portal, early-warning service, simulator
ii	NLP and classification	TF-IDF + logistic regression (or Claude with JSON schema) → intents; regex and database-grounded entity extraction
iii	Workflow identification	Each intent maps to a workflow template of agent steps
iv	Agent selection	Templates are merged, de-duplicated and ordered topologically into dependency levels
v	Coordination	Agents run level by level, post findings to the blackboard and exchange FIPA-ACL messages
vi	Constraint analysis	CP-SAT allocation, exclusion of held resources, conflict detection and resolution
vii	Recommendations	Recommendation and ProposedAction objects with factor-level explanations
viii	Human approval	Autonomy policy decides which actions need approval; approvals routed to authorised roles
ix	Execution	Executor applies approved or low-risk actions transactionally
x	Logging & monitoring	Trace, audit log, notifications, live WebSocket updates and a coordination brief

4.3 Agent Catalogue

Table 4.2: Specialised agents

Agent	Responsibility	Key techniques	Typical actions
Bed Management	Real-time bed tracking and matching	Hard/soft constraint scoring, held-bed exclusion, portable-equipment pairing	Reserve bed
Appointment & Queue	Slots, queues, waiting time	Slot search, ageing priority, M/M/c Erlang-C	Book appointment

Agent	Responsibility	Key techniques	Typical actions
Staff Scheduling	Doctor/nurse matching, rosters	Skill/workload scoring, CP-SAT rostering	Publish roster, redeploy staff
Diagnostic Coordination	Tests and modalities	Protocol workups, least-loaded modality scheduling, TAT tracking	Create diagnostic orders
Pharmacy	Medication safety and stock	Allergy cross-reactivity, interaction and dose rules, reorder quantity	Create prescription, purchase order
Patient Care Coordinator	Patient identity and journey	Care pathways, readiness scoring	Create pathway, discharge
Emergency Support	Triage and mobilisation	NEWS2, red-flag detection, surge detection	Assign ER bay and team
Resource Optimization	Global allocation and planning	CP-SAT bed & theatre optimisation, Holt-Winters, GBM LOS	Admit/transfer, book surgery, apply schedules
Notification	Communication	Role/user routing, channel selection	Alerts, approval requests, patient SMS

4.4 Agent Interaction

Figure 4.3 shows the interaction for the synopsis example “Patient needs admission and requires ICU monitoring”. The orchestrator identifies the admission intent and the ICU/ventilator requirement, the Patient Care Agent resolves (or provisionally registers) the patient, the Bed and Staff agents propose candidates to the Resource Optimizer, which solves the joint allocation while considering patients already waiting, and the Notification Agent requests approval from a doctor.

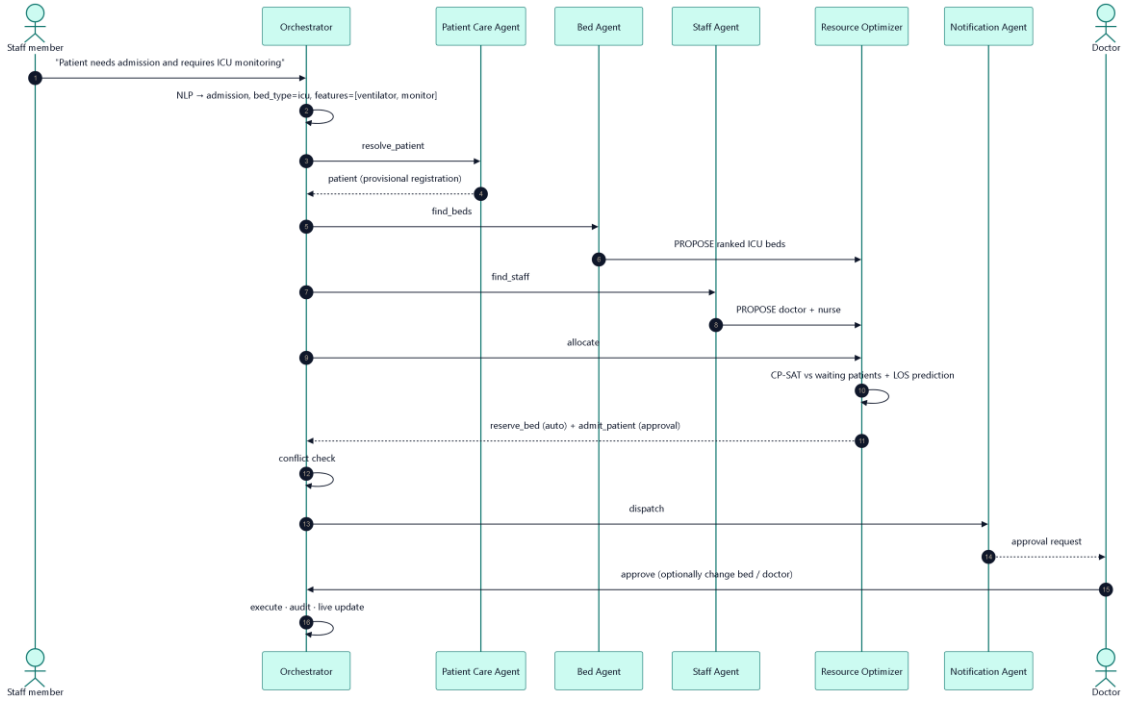


Figure 4.3: Sequence diagram for an ICU admission request

4.5 Human-in-the-Loop Design

Agents only propose actions. The orchestrator applies the configured autonomy policy to every proposal (Figure 4.4). Reversible, low-risk actions such as holding a bed or booking an appointment execute immediately in the default balanced mode; actions that affect clinical care require approval from the roles listed in Table 4.4. Approvers can modify the AI's choice (bed, doctor, time, dose) before approving; rejecting releases held resources and cancels pending tasks.

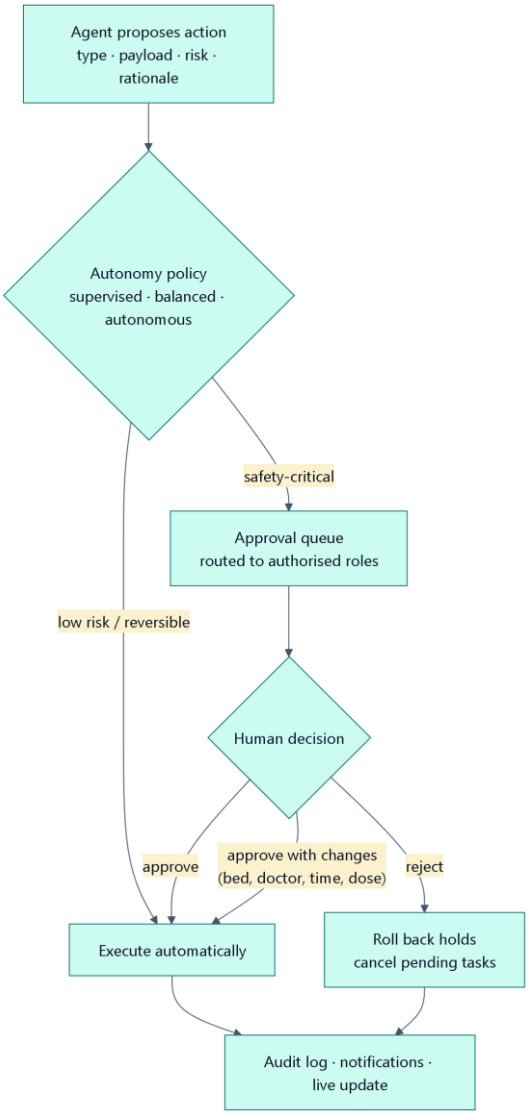


Figure 4.4: Human-in-the-loop decision gate

Table 4.3: Human-in-the-loop policy (balanced mode)

Action	Risk	Balanced mode	Approver roles
Hold a bed	Low	Automatic	—
Book OPD appointment	Low	Automatic	—
Admit / transfer patient	Medium–High	Approval	Doctor, Admin
Assign ER bay and team	High–Critical	Approval	Doctor, Admin
Investigations requested by a prescriber	Low	Automatic	—
AI-suggested protocol workup	Medium	Approval	Doctor, Admin
Prescription with major/contraindicated alert	High–Critical	Approval (override)	Doctor, Admin
Purchase order	Low	Approval	Pharmacist, Doctor, Admin
Book surgery / apply theatre schedule	Medium–High	Approval	Doctor, Admin

Action	Risk	Balanced mode	Approver roles
Discharge	Medium–High	Approval	Doctor, Admin
Roster publication / staff redeployment	Medium	Approval	Admin
Global bed placement	Medium	Approval	Admin, Nurse, Receptionist

Table 4.4: Configurable autonomy modes

Autonomy mode	Behaviour
Supervised	Every agent action waits for human approval.
Balanced (default)	Low-risk reversible actions execute automatically; clinical and risky actions need approval.
Autonomous	Low and medium-risk operational actions execute automatically; high/critical-risk actions, prescribing, discharge, staffing changes and AI-suggested workups still need approval.

4.6 Ward Early-Warning Escalation

Each observation set is scored with NEWS2 and the response prescribed by the Royal College of Physicians is applied (Figure 4.5). A score of seven or more triggers an emergency alert and — for patients outside intensive care — an automatic orchestrator request to transfer the patient to the ICU, which is then planned by the agents and approved by a doctor.

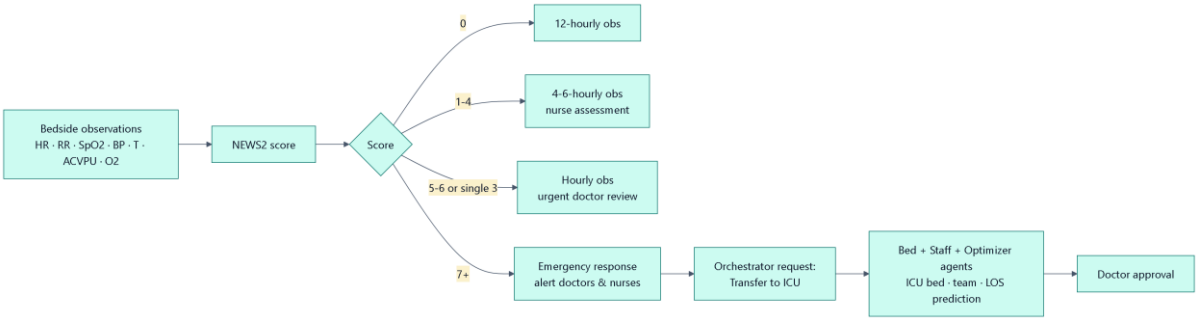


Figure 4.5: NEWS2 escalation workflow

4.7 Data Flow Diagrams

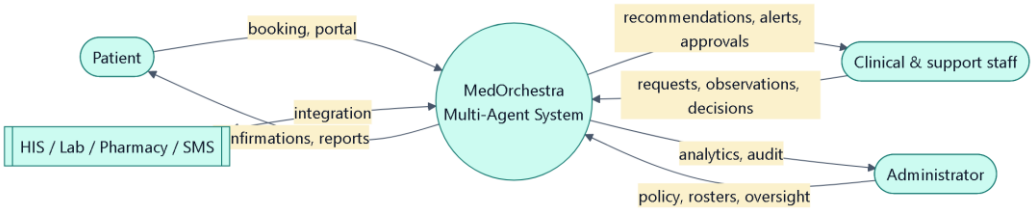


Figure 4.6: Context-level (Level 0) data flow diagram

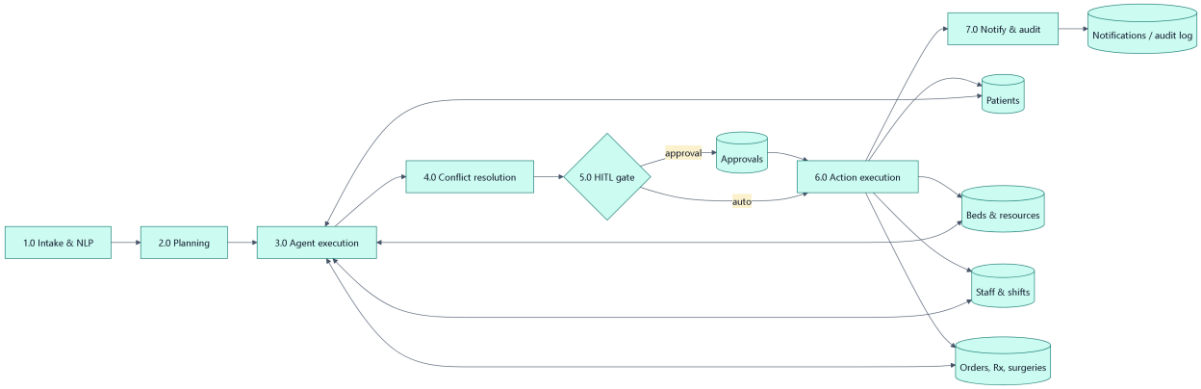


Figure 4.7: Level 1 data flow diagram

4.8 Use Case Diagram

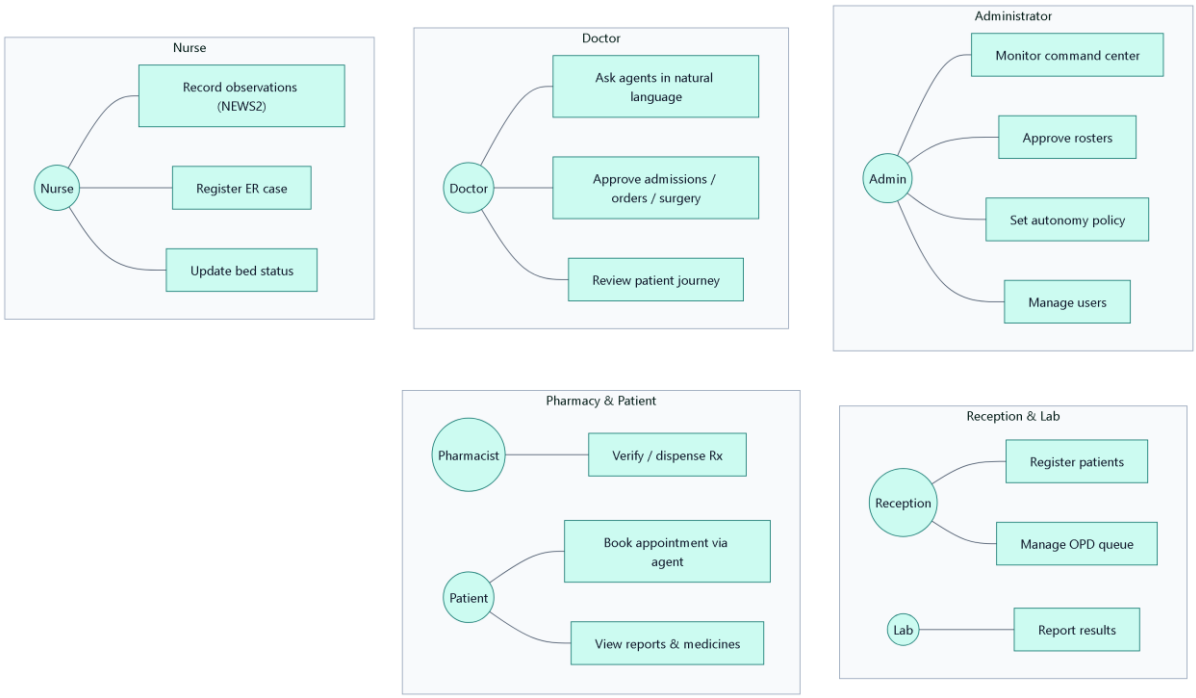


Figure 4.8: Use cases by actor

4.9 Database Design

The data model links every operational event to a patient, a resource and a workflow (Figure 4.9). Table 4.6 summarises the main tables.

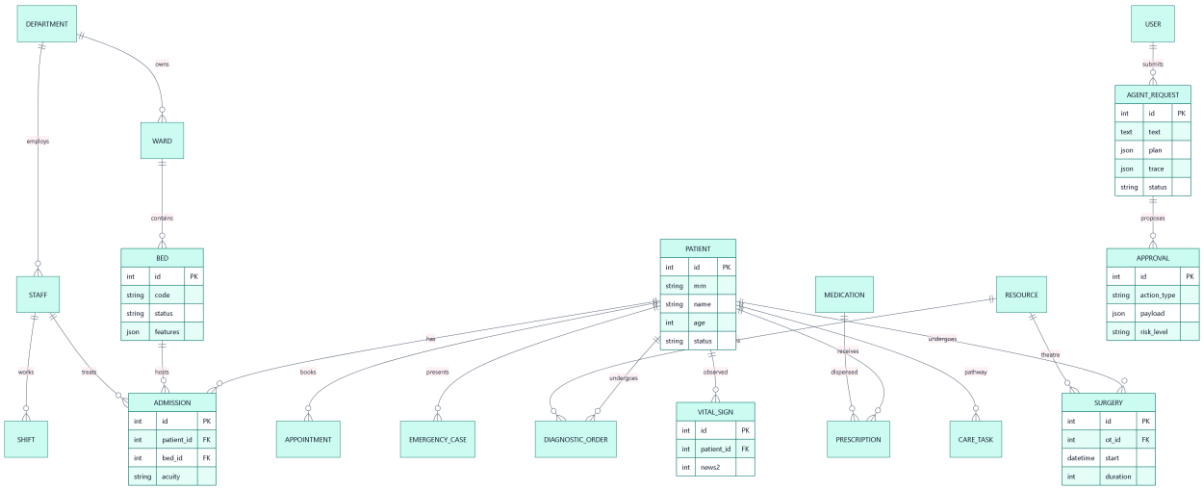


Figure 4.9: Entity-relationship diagram (main entities)

Table 4.5: Database tables

Table	Purpose
users, departments, staff, shifts	Accounts and roles; departments; clinicians with skills, workload and duty status; roster shifts
patients	Demographics, allergies, conditions, status and clinical risk score
wards, beds, resources	Wards and beds with type, features and status; theatres and equipment with usage and maintenance data
admissions, emergency_cases	Inpatient episodes with bed, team, acuity and expected discharge; ED cases with vitals, NEWS2 and triage level
appointments, diagnostic_orders, prescriptions, medications	OPD bookings and queues; lab/imaging orders and results; prescriptions with safety alerts; formulary and stock
vital_signs, surgeries, care_tasks	Bedside observations with NEWS2; theatre bookings; care-pathway steps
agent_requests, approvals	Orchestrated requests with intents, plan, trace, results and conflicts; human-in-the-loop decisions
notifications, audit_logs, settings, daily_stats	Alerts; audit trail; autonomy policy; historical metrics for analytics and forecasting

4.10 API Design

Table 4.6: REST API overview (interactive documentation at /docs)

Endpoint group	Examples
Authentication & users	POST /api/auth/login · GET /api/auth/me · POST /api/auth/change-password · /api/users
AI agents & approvals	POST /api/agents/requests · GET /api/agents/requests/{id} · GET /api/approvals · POST /api/approvals/{id}/decide
Patients & ward	GET /api/patients · GET /api/patients/{id} · POST /api/patients/{id}/vitals · GET /api/admissions · GET /api/admissions/{id}/summary
Resources & theatres	GET /api/wards · PATCH /api/beds/{id} · GET /api/surgeries · GET /api/ot/slots · POST /api/ot/optimize

Endpoint group	Examples
Clinical workflows	/api/appointments · /api/queue · /api/diagnostics · /api/prescriptions · /api/emergency · /api/emergency/triage-preview
Insights & system	/api/dashboard · /api/dashboard/mywork · /api/analytics/* · /api/search · /api/settings · /api/audit · /api/simulation/*
Real time	WebSocket /ws?token=… (agent traces, notifications, approvals, refresh events)

Chapter 5: Implementation

5.1 Technology Stack

Table 5.1: Technology stack

Layer	Technology	Purpose
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4, Recharts, lucide icons	Role-based single-page application and charts
Mobile	Progressive Web App (manifest, service worker, installable)	Access from phones and tablets
Backend	Python 3.11, FastAPI, Pydantic v2, Uvicorn	REST API, validation, WebSockets
Database	SQLAlchemy 2 ORM — SQLite (development), PostgreSQL 16 (Docker)	Persistent storage
AI / NLP	scikit-learn TF-IDF + logistic regression; optional Claude (claude-opus-5) with structured outputs	Intent and entity understanding, coordination briefs
Optimisation	Google OR-Tools CP-SAT; SciPy (Hungarian fallback)	Bed allocation, rostering, theatre scheduling
Analytics / ML	NumPy Holt-Winters; scikit-learn Gradient Boosting	Forecasting and length-of-stay prediction
Security	PyJWT (HS256), PBKDF2-SHA256, RBAC	Authentication and authorisation
Deployment & testing	Docker Compose (PostgreSQL, API, Nginx); pytest; TypeScript compiler	Reproducible deployment and quality assurance

5.2 Project Structure

```
backend/app/
  agents/     orchestrator.py, base.py (blackboard, messages), nlp.py, llm.py, executor.py
(HITL)
              bed_, appointment_, staff_, diagnostic_, pharmacy_, care_, emergency_,
              optimization_, notification_agent.py
  services/   optimizer.py, ot_scheduler.py (OR-Tools) · triage.py, early_warning.py
(NEWS2)
              los_model.py (ML) · forecasting.py · queueing.py · maintenance.py ·
clinical.py
              settings_store.py (autonomy policy) · activity.py (audit, notifications)
  routers/    auth, users, dashboard, patients, ward, resources, surgery, appointments,
staff,
              clinical, agents, insights
  models.py · serializers.py · seed.py · simulation.py · main.py
backend/tests/   test_services.py · test_api.py · test_extended.py (38 tests)
frontend/src/    pages/ (20 screens) · components/ (layout, agent network, approval card,
UI kit)
docs/            ARCHITECTURE.md · DEMO_SCRIPT.md          docker-compose.yml
```

5.3 Orchestrator

The orchestrator (`agents/orchestrator.py`) processes each request in a worker thread. After understanding, it builds a plan from intent templates: each step is an `agent.action` pair with dependencies, steps shared by several intents are merged, and a topological level is computed

so that independent steps are dispatched together. An `AgentContext` object acts as the blackboard: agents post findings such as `bed_candidates`, `doctor_candidates`, `triage` or `allocation`, and read those of other agents. Agents communicate with FIPA-ACL performatives (for example the Bed Agent PROPOSEs candidates to the Optimizer, the Emergency Agent REQUESTs a team from the Staff Agent, the Pharmacy Agent REFUSEs a contraindicated drug). Every event is appended to a trace and broadcast over WebSockets.

After all agents have run, the conflict resolver checks the proposed actions for resource contention with higher-priority patients, staff overload, missing resuscitation capacity, safety refusals and duplicate effects, and applies priority rules. The human-in-the-loop gate then applies the autonomy policy, executes permitted actions and creates approval records for the rest. Allocation transactions are serialised with a lock, preventing two concurrent requests from double-booking a bed.

5.4 Natural-Language Understanding

The local pipeline trains, at start-up, a logistic-regression classifier over a union of word (1–2-gram) and character (3–5-gram) TF-IDF features using 12 intents and about 220 template sentences. Multi-intent requests are handled by classifying each clause and by word-bounded keyword rules; suppression rules prevent hospital-level questions (e.g. “forecast admissions”) from triggering patient workflows. Entities are extracted with regular expressions and grounded against the database: patient names and MRNs, medicines, vital signs (with °F→°C conversion), bed types and features, tests, surgical procedures, departments, dates, times and staff counts. When an Anthropic API key is configured, Claude is called with a strict JSON schema at low effort and its output is merged with the database-grounded entities; any failure falls back to the local pipeline.

5.5 Specialised Agents

Patient Care Coordinator Agent

Resolves the patient from an explicit reference, MRN or name; when an admission or emergency concerns an unidentified patient it creates a provisional registration so that care is not delayed. It creates care pathways (admission, emergency, diagnostic, OPD, discharge, surgical), reports journey status with overdue tasks and turnaround breaches, and scores discharge readiness.

Bed Management Agent

Derives bed requirements from the request, triage level, clinical protocol and patient age; ranks free beds by hard constraints (type, features) and soft costs; excludes beds held by other pending approvals; and, when no ventilator-equipped bed is free, pairs a monitored bed with a portable ventilator. It also predicts beds that will free up from planned discharges.

Staff Scheduling Agent

Ranks doctors by department, specialisation, duty status, workload and seniority, and nurses by department, workload and skills (critical care, paediatrics, triage). It pages specialist consultants for emergencies, proposes redeployments that keep minimum cover elsewhere, and generates weekly rosters with CP-SAT.

Diagnostic Coordination Agent

Maps requested tests to catalogue codes and modalities, suggests protocol-based workups (e.g. acute coronary syndrome, stroke, sepsis) for clinician review, schedules each test on the least-loaded available machine according to priority, and tracks turnaround time.

Pharmacy Agent

Screens medicines for allergy cross-reactivity, drug–drug interactions against active prescriptions, maximum single dose and stock; proposes alternatives from the same drug class; and computes purchase-order quantities as 30-day demand plus safety stock minus current stock.

Emergency Support Agent

Computes NEWS2 and detects red-flag complaints to assign one of five triage levels with target times, creates the emergency case, requests capacity and a team, and monitors ED load for surge conditions and target-time breaches.

Appointment & Queue Agent

Searches free slots for every doctor in the department, ranks options by delay and queue length, predicts department waiting time with an M/M/c model at peak-hour load, and schedules post-discharge follow-ups.

Resource Optimization Agent

Performs the joint allocation of bed, doctor and nurse using CP-SAT against all waiting patients, predicts length of stay, books surgeries and re-optimises theatre days, forecasts demand and detects bottlenecks across the hospital.

Notification Agent

Routes critical emergency alerts, approval requests, receiving-nurse notices, patient SMS confirmations and capacity alerts to the right roles and users over in-app, SMS and e-mail channels (the latter two simulated).

5.6 Algorithms

NEWS2 and triage

Table 5.2: NEWS2 scoring (Royal College of Physicians, 2017)

Parameter	3	2	1	0	1	2	3
Respiration rate (/min)	≤8		9–11	12–20		21–24	≥25
SpO_2 scale 1 (%)	≤91	92–93	94–95	≥96			
Supplemental O_2		Yes		No			
Systolic BP (mmHg)	≤90	91–100	101–110	111–219			≥220
Pulse (/min)	≤40		41–50	51–90	91–110	111–130	≥131
Consciousness				Alert			C/V/P/U
Temperature (°C)	≤35.0		35.1–36.0	36.1–38.0	38.1–39.0	≥39.1	

The aggregate score maps to low (0–4), low-medium (any single parameter scoring 3), medium (5–6) and high (≥7) clinical risk. Triage level 1 is assigned for NEWS2 ≥ 7, unresponsiveness or critical red flags; level 2 for NEWS2 5–6, a single parameter scoring 3 or emergent red flags such as chest pain or stroke signs; level 3 for other abnormal scores or urgent complaints; levels 4–5 for minor presentations. Age ≥ 75 years or ≤ 1 year escalates one level.

Bed allocation with CP-SAT

For patients i and beds j, a Boolean variable x[i,j] exists only for feasible pairs (allowed bed type and all required features). The model is:

$$\text{maximise } \Sigma\, x[i,j] \cdot (1000 \cdot (6 - \text{priority}_i) - 10 \cdot \text{cost}[i,j])$$

$$\text{subject to } \Sigma_j\, x[i,j] \le 1 \;\; \forall\, i, \quad \Sigma_i\, x[i,j] \le 1 \;\; \forall\, j$$

$$\text{cost} = 12 \cdot \text{type rank} + 8 \cdot [\text{other department}] + 2 \cdot |\Delta\text{floor}| + 15 \cdot \text{ward occupancy} + 20 \cdot (1 - \text{nurse coverage}) + 6 \cdot \text{missing preferred} + 4 \cdot \text{wasted premium equipment}$$

The dominant priority term guarantees that the most urgent patients are placed first; the cost terms produce an explainable, load-balanced choice that also keeps scarce equipment for patients who need it.

Staff rostering with CP-SAT

Variables x[s,d,t] indicate that staff member s works shift t (morning, evening, night) on day d. Constraints: at most one shift per day; no night shift followed by a morning shift; at most five shifts per week; approved leave; and coverage Σ_s x[s,d,t] + shortfall ≥ requirement per department, role and shift. The objective minimises 1000·Σ shortfall + 25·(night-shift spread) + 25·(total-shift spread) + Σ x, i.e. coverage first, then fairness, then no over-staffing.

Operating-theatre scheduling

The slot finder evaluates, for each theatre, the requested earliest start and the end of every busy block; a start is feasible if the interval [start, end + 30 min turnover) overlaps no booking in that theatre, the surgeon is free and an elective case ends by 20:00. The day optimiser uses CP-SAT interval scheduling: for each not-yet-started case s and theatre o an optional interval with presence p[s,o], exactly one presence per case, NoOverlap constraints per theatre (including cases already running) and per surgeon, and the objective

$$\min \; \Sigma\, w_priority \cdot start_s + 40 \cdot \Sigma\, overtime_s + 3 \cdot makespan + 20 \cdot \Sigma\, theatre\ changes$$

with weights 30, 6 and 1 for emergency, urgent and elective cases.

Waiting-time prediction (M/M/c)

With arrival rate λ (peak-hour arrivals), service rate μ = 60 / consultation minutes and c doctors on duty, the offered load is a = λ/μ and utilisation ρ = a/c. The probability of waiting and the mean queueing delay are:

$$P(wait) = [a^c / (c!\,(1-\rho))] / [\,\Sigma_{k=0}\text{^\{c−1\}}\, a^k/k! + a^c/(c!\,(1-\rho))\,], \quad Wq = P(wait) / (c\mu - \lambda)$$

Within the queue, patients are ordered by a dynamic priority score 20·(6 − priority) + 0.4·min(waited, 180) + bonuses for seniors, young children and lateness; the waiting-time term (ageing) prevents starvation of low-priority patients.

Demand forecasting (Holt-Winters)

$$\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1}), \quad b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta) b_{t-1}, \quad s_t = \gamma(y_t - \ell_t) + (1-\gamma) s_{t-m}$$

$$\hat{y}_{t+h} = \ell_t + h \cdot b_t + s_{t+h-m}, \quad m = 7$$

The smoothing constants are selected by grid search on one-step-ahead squared error; a 95 % prediction interval is derived from the residual standard deviation and widens with the horizon.

Length-of-stay prediction

A GradientBoostingRegressor (180 trees, depth 3, learning rate 0.06) is trained on age, acuity, number of comorbidities, emergency admission and ward type. Two additional models with

quantile loss ($\alpha = 0.1$ and 0.9) provide an 80 % prediction interval. The Resource Optimization Agent uses the prediction to set the expected discharge date and states it, with the interval, in the approval rationale.

Predictive maintenance and drug safety

Equipment maintenance risk combines calendar age since the last service and usage-hour progress through the manufacturer service interval (risk = 0.65·days/interval + 0.35·hour progress), flagging high-risk devices as a bottleneck. Drug safety uses a curated knowledge base of allergy classes (e.g. penicillins, sulphonamides, NSAIDs), fifteen clinically significant interactions and maximum single doses.

5.7 Security, Audit and Real-Time Updates

Passwords are stored as salted PBKDF2-SHA256 hashes; users receive a signed JWT; every endpoint declares the roles allowed to call it; approvals are additionally checked against the required approver roles. Every login, data change, approval decision and agent action is written to the audit log with actor, action, entity and details. A WebSocket event bus pushes agent traces, notifications, approvals and refresh events so that all dashboards update live.

5.8 Frontend and Progressive Web App

The frontend has twenty screens including the command centre with role-specific work lists, the AI Agent Console (with an animated agent network, pipeline progress, live trace, understanding and plan panels, explainable findings and inline approvals), approvals, emergency board, inpatient census, patient record with vitals trend, beds and equipment, operation-theatre timeline, OPD queues, staff rosters, diagnostics, pharmacy, analytics, audit log, users, system and policy, printable discharge summaries and the patient portal. A global command palette (Ctrl + K) searches patients, staff, beds and medicines, and the agent console accepts voice input. The application is installable as a PWA.

5.9 Synthetic Data and Live Simulation

A deterministic generator creates a 108-bed hospital with ten wards, fourteen departments, 86 staff, 320 patients, 67 active admissions with 402 bedside observations, emergency cases, 190 OPD appointments for the current day, diagnostic orders, prescriptions, 17 theatre cases and 180 days of historical statistics with weekly and monsoon seasonality. A live simulator generates walk-ins, consultations, laboratory results, bed turnover, theatre progress, routine

observations, deterioration events (which trigger ICU-transfer proposals) and emergency arrivals routed through the full agent pipeline.

Chapter 6: Results and Screenshots

This chapter presents the working system. All screenshots were captured from the running application with the synthetic hospital data.

6.1 Login and Role-Based Access

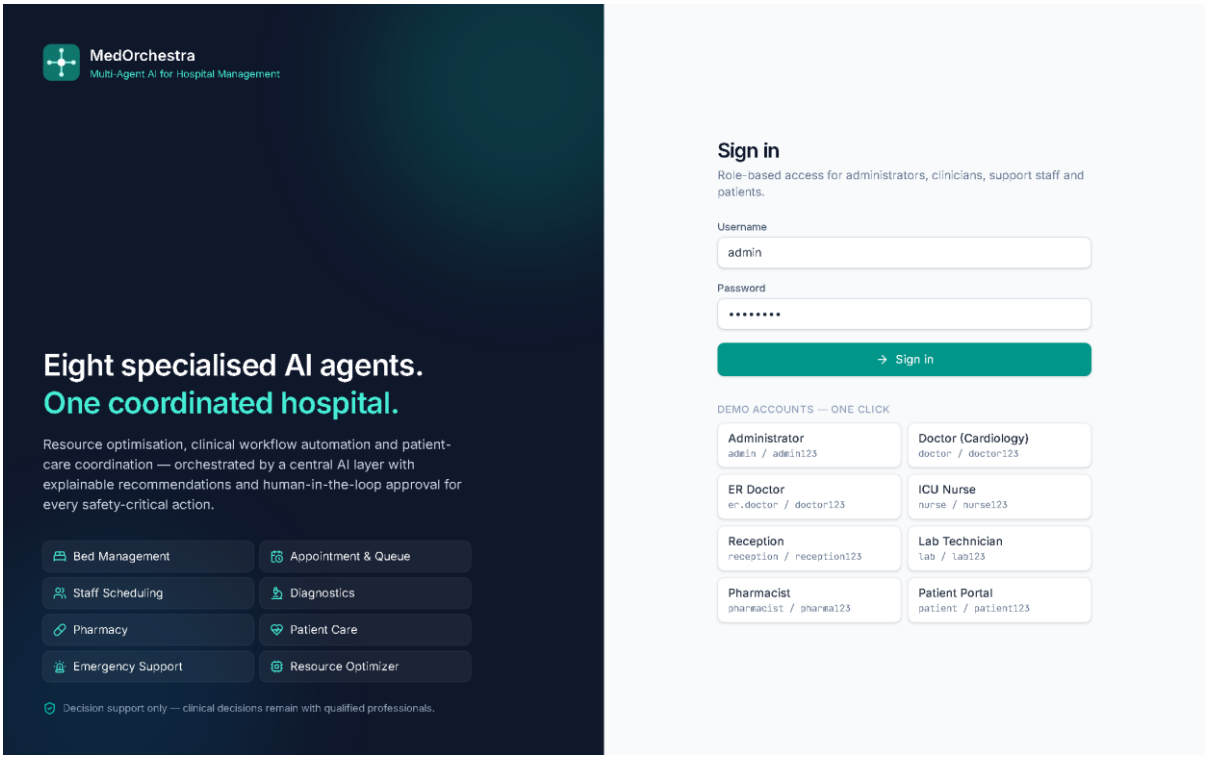


Figure 6.1: Login screen with one-click demo accounts for each role

6.2 Command Center

The command centre shows role-specific work lists (here, inpatients ordered by NEWS2 and the theatre list), live KPIs, ward occupancy, the emergency board, patient-flow trends and bottlenecks detected by the Resource Optimization Agent.

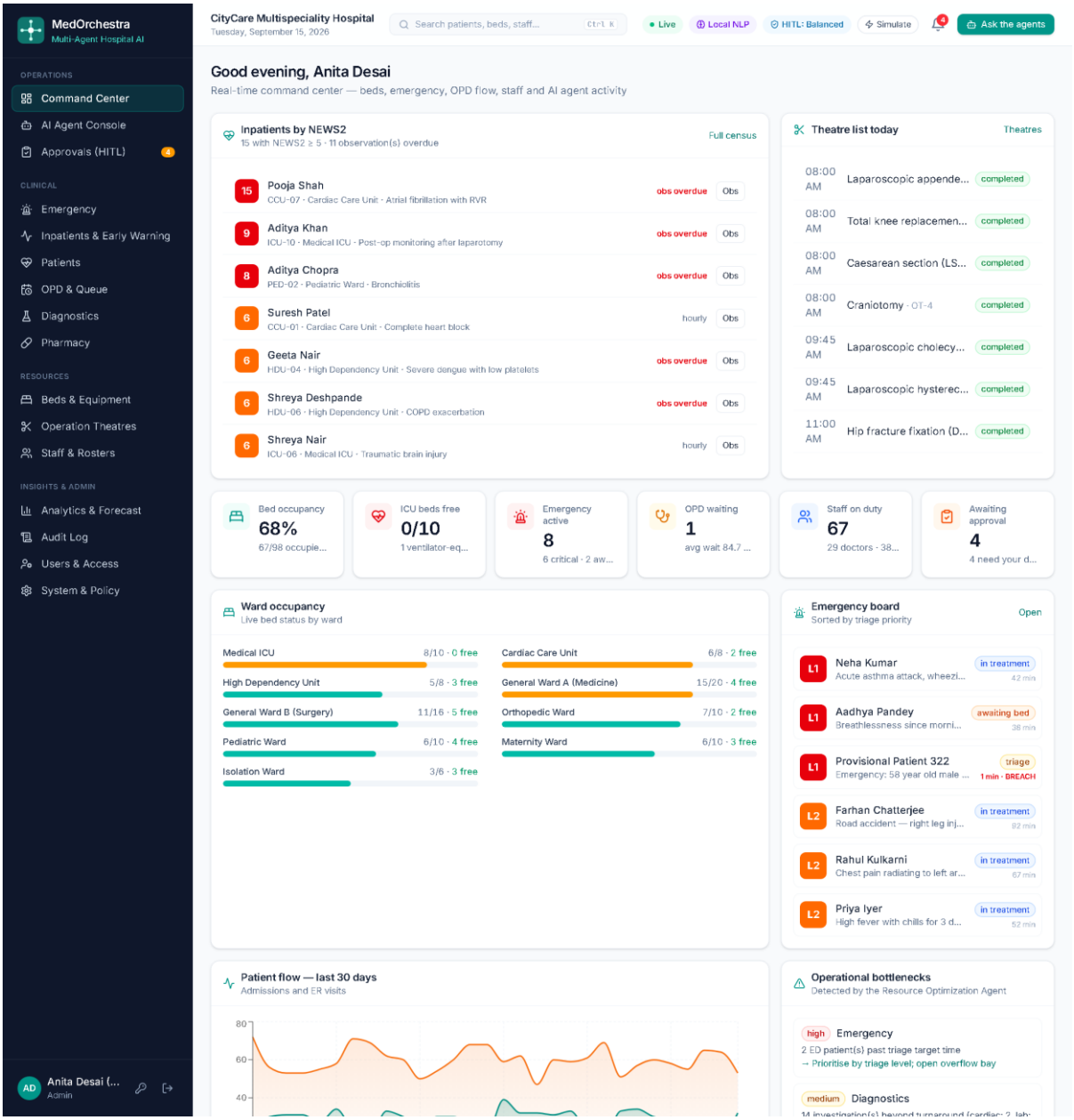


Figure 6.2: Command center (administrator)

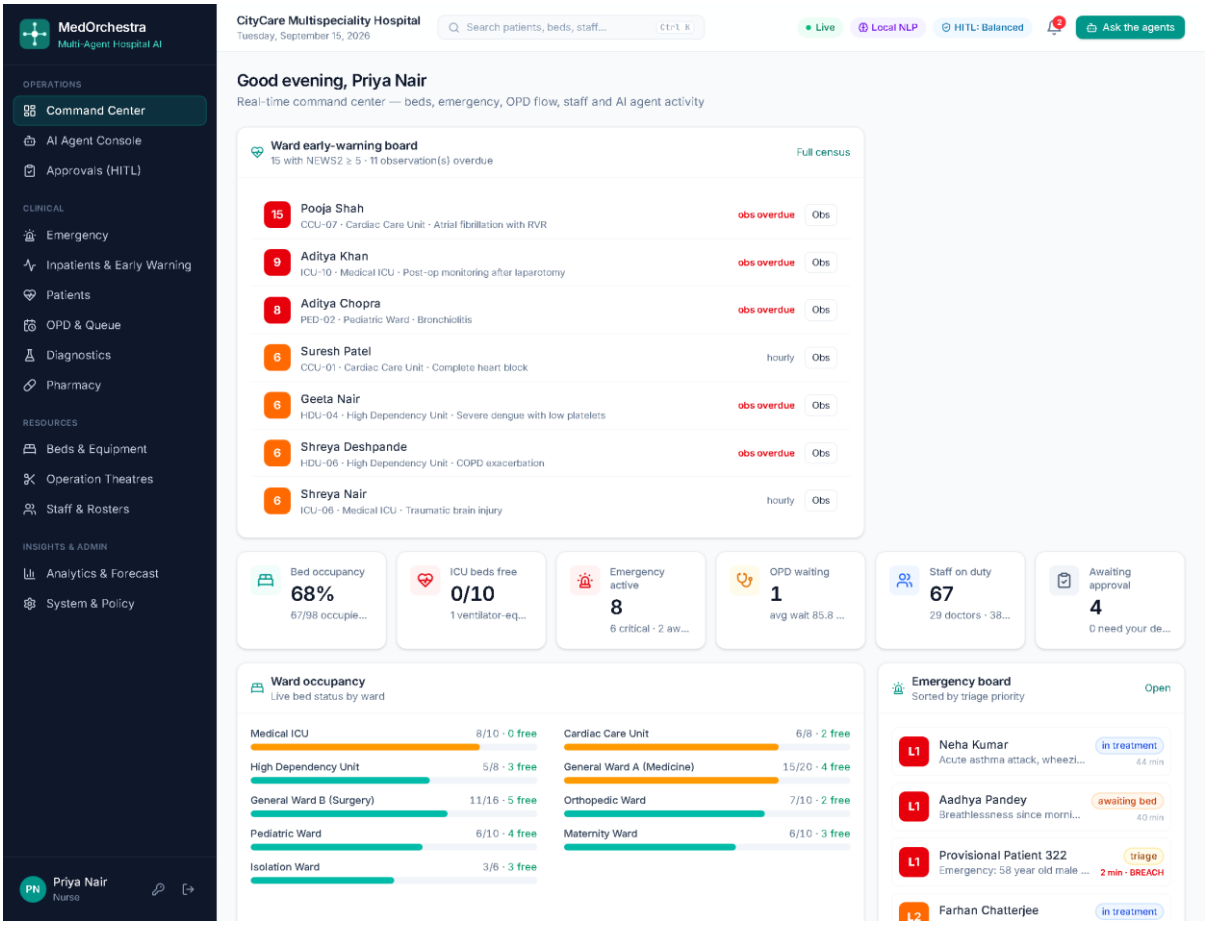


Figure 6.3: Nurse view: ward early-warning board with overdue observations

6.3 AI Agent Console

For the synopsis example “Patient needs admission and requires ICU monitoring with ventilator support”, the console shows the live agent network with the agents that participated, the pipeline progress, the live trace of messages and results, the understanding panel (admission intent, ICU and ventilator requirements) and the execution plan.

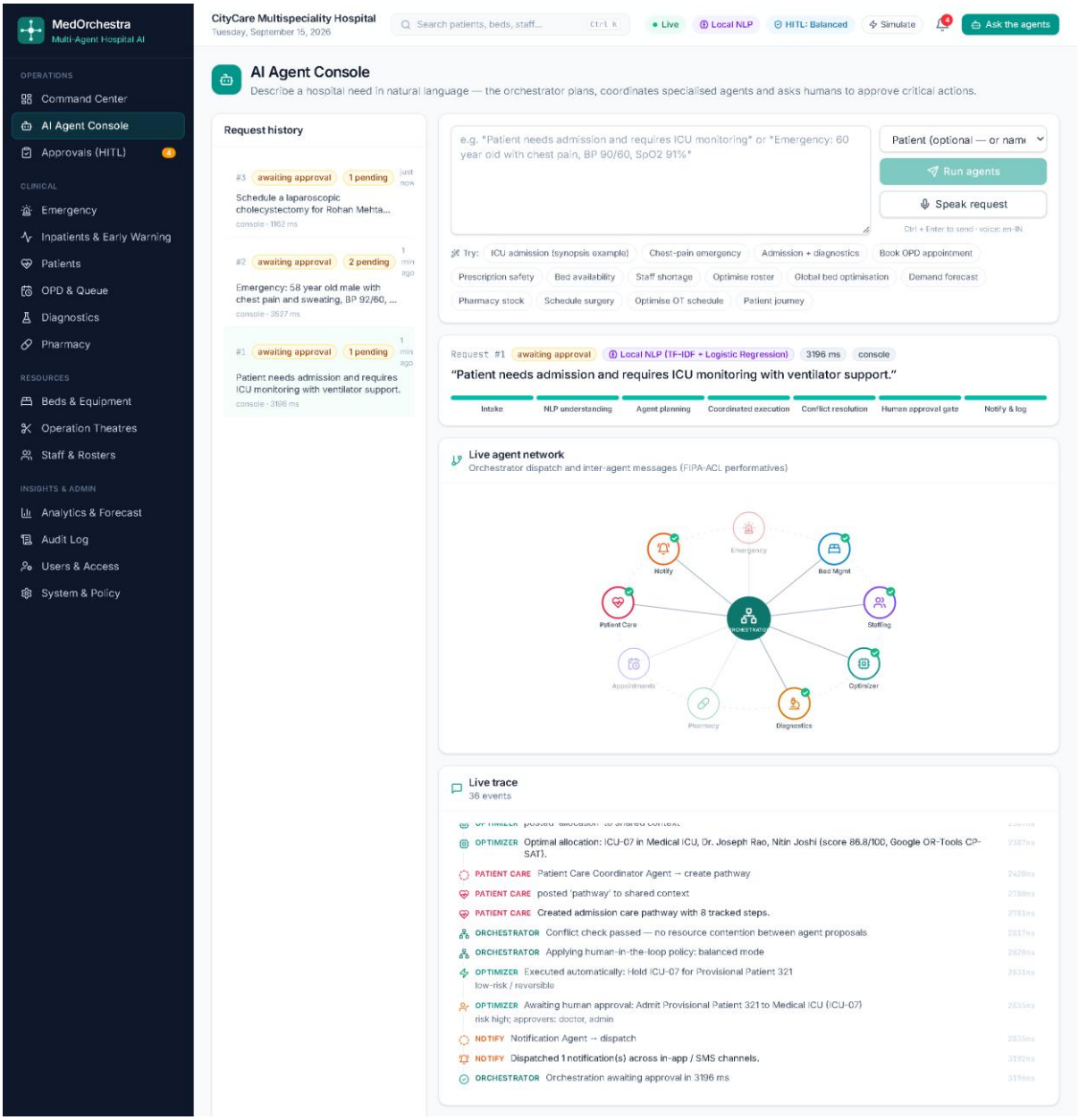


Figure 6.4: AI Agent Console — live agent network, trace, understanding and plan

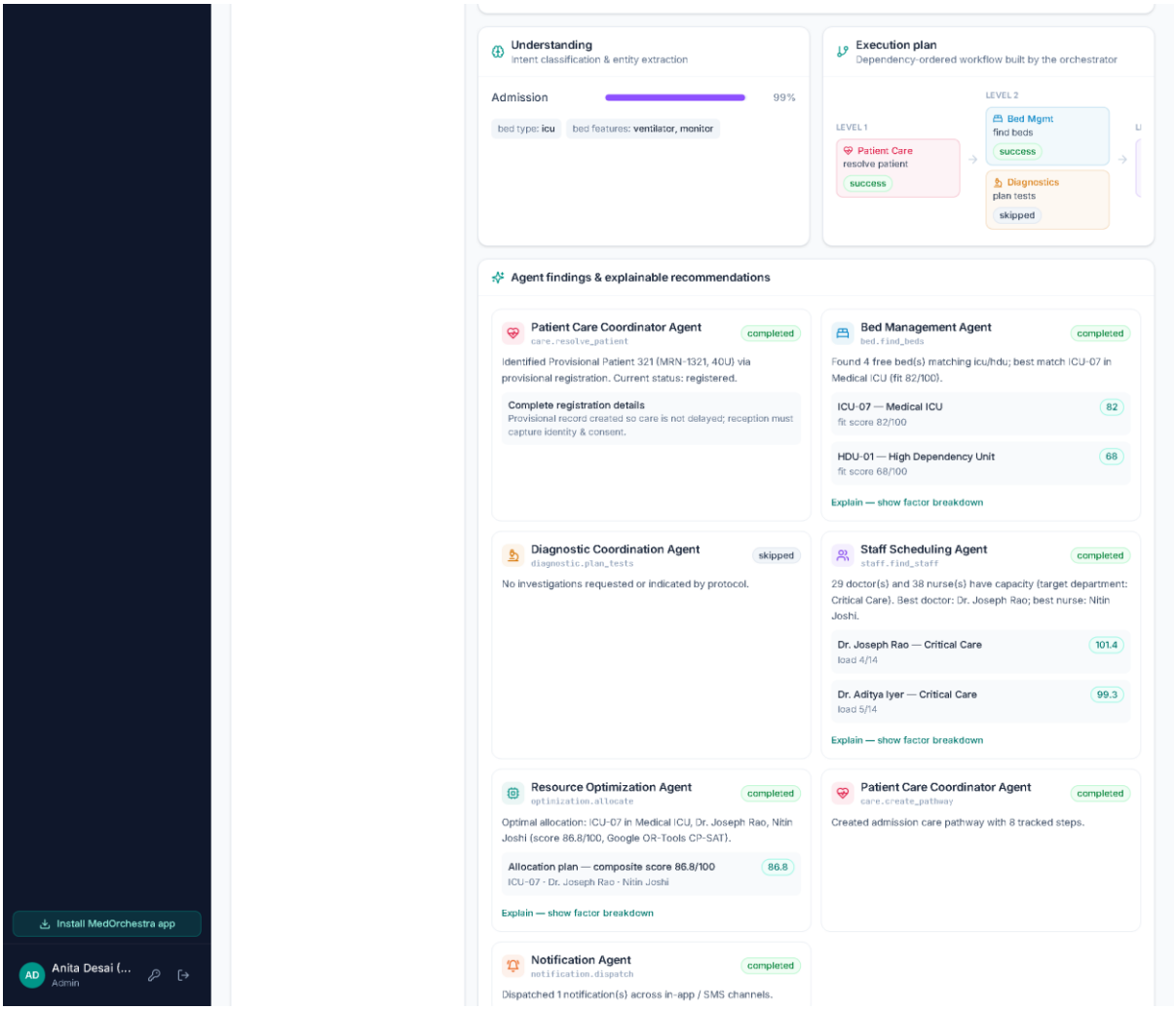


Figure 6.5: Agent findings with explainable recommendations and the approval request

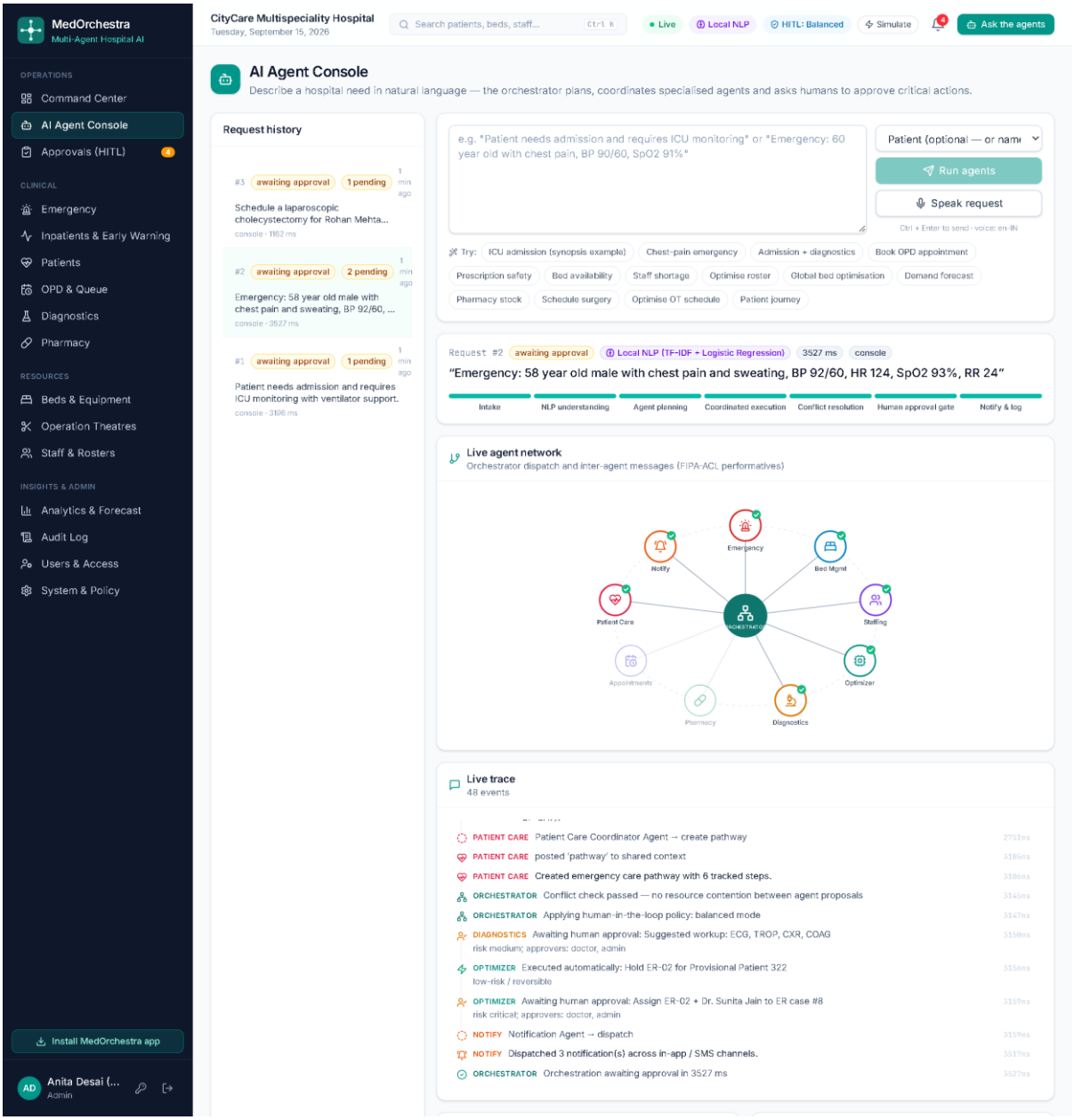


Figure 6.6: Emergency request: NEWS2 triage, resuscitation bay, ED team and protocol workup

6.4 Human-in-the-Loop Approvals

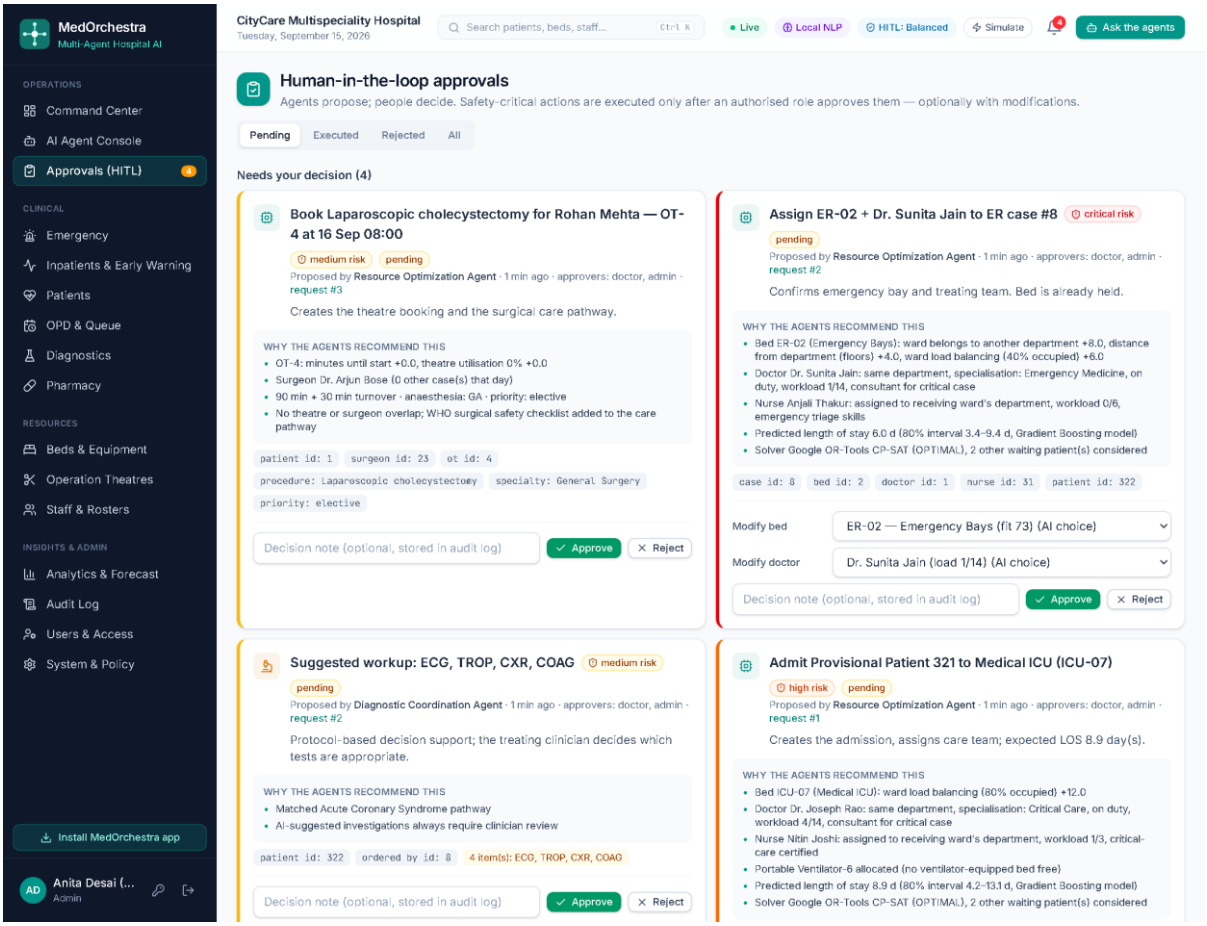


Figure 6.7: Approval queue with risk levels, rationale and the option to modify the AI's choice

6.5 Emergency Department

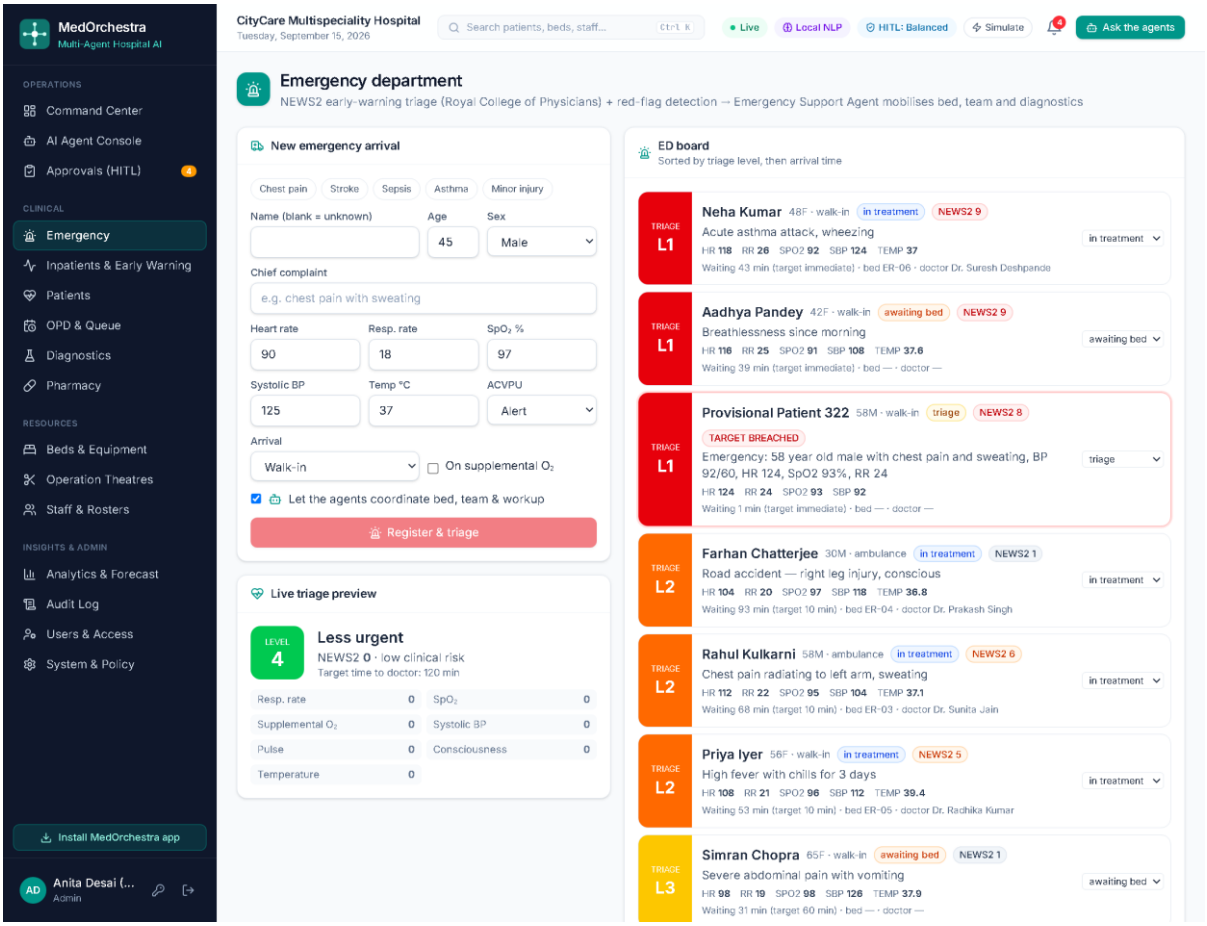


Figure 6.8: ED board sorted by triage level, with live NEWS2 preview for new arrivals

6.6 Inpatients and Patient Record

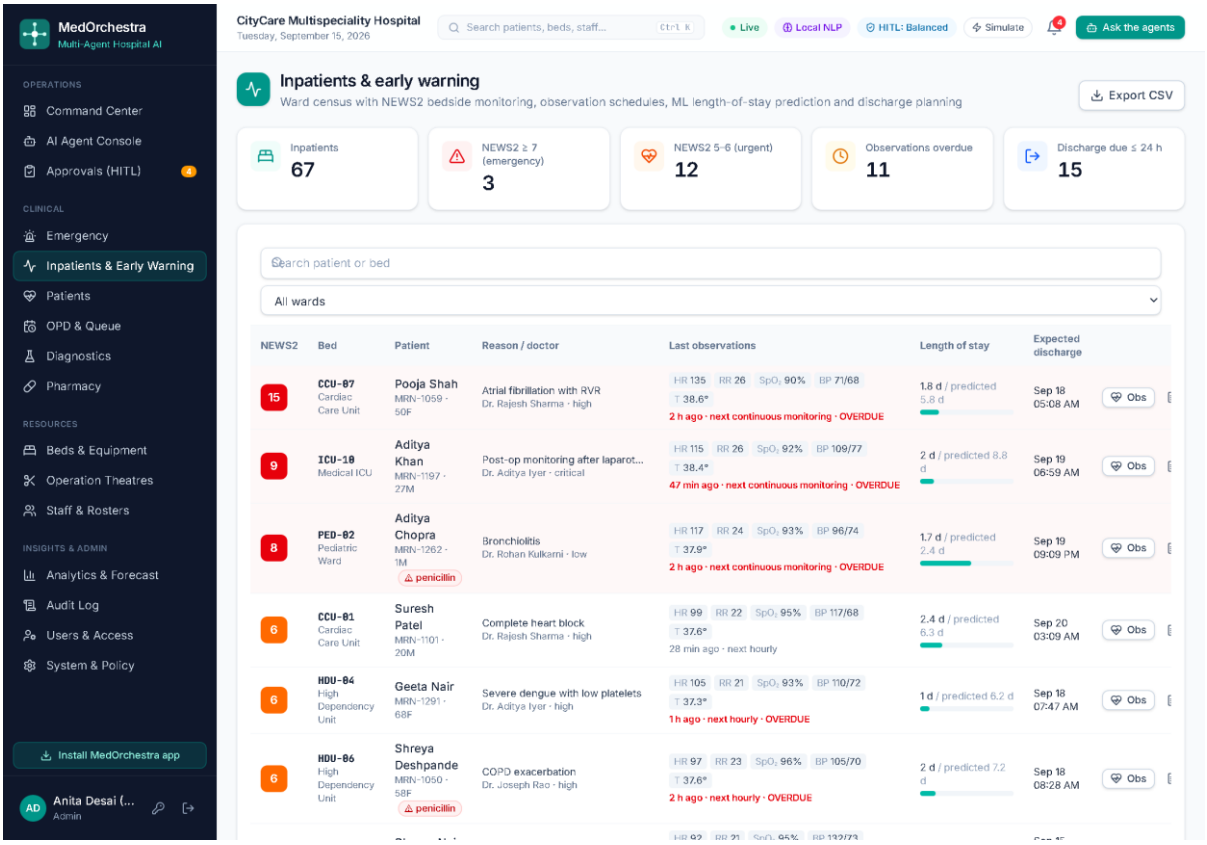


Figure 6.9: Inpatient census with NEWS2, observation schedule and ML-predicted length of stay

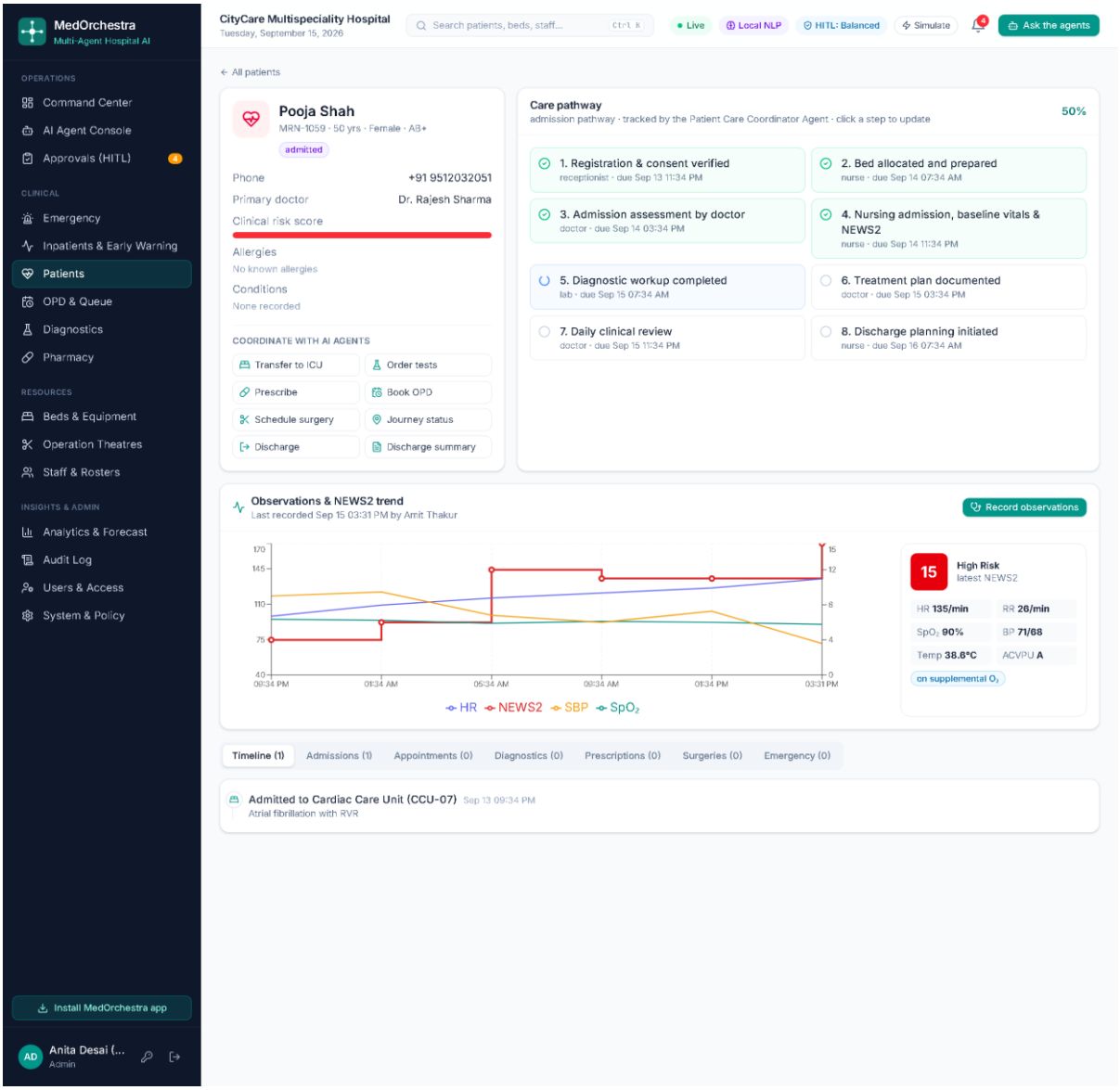


Figure 6.10: Patient record: care pathway, quick agent actions and NEWS2 / vital-signs trend

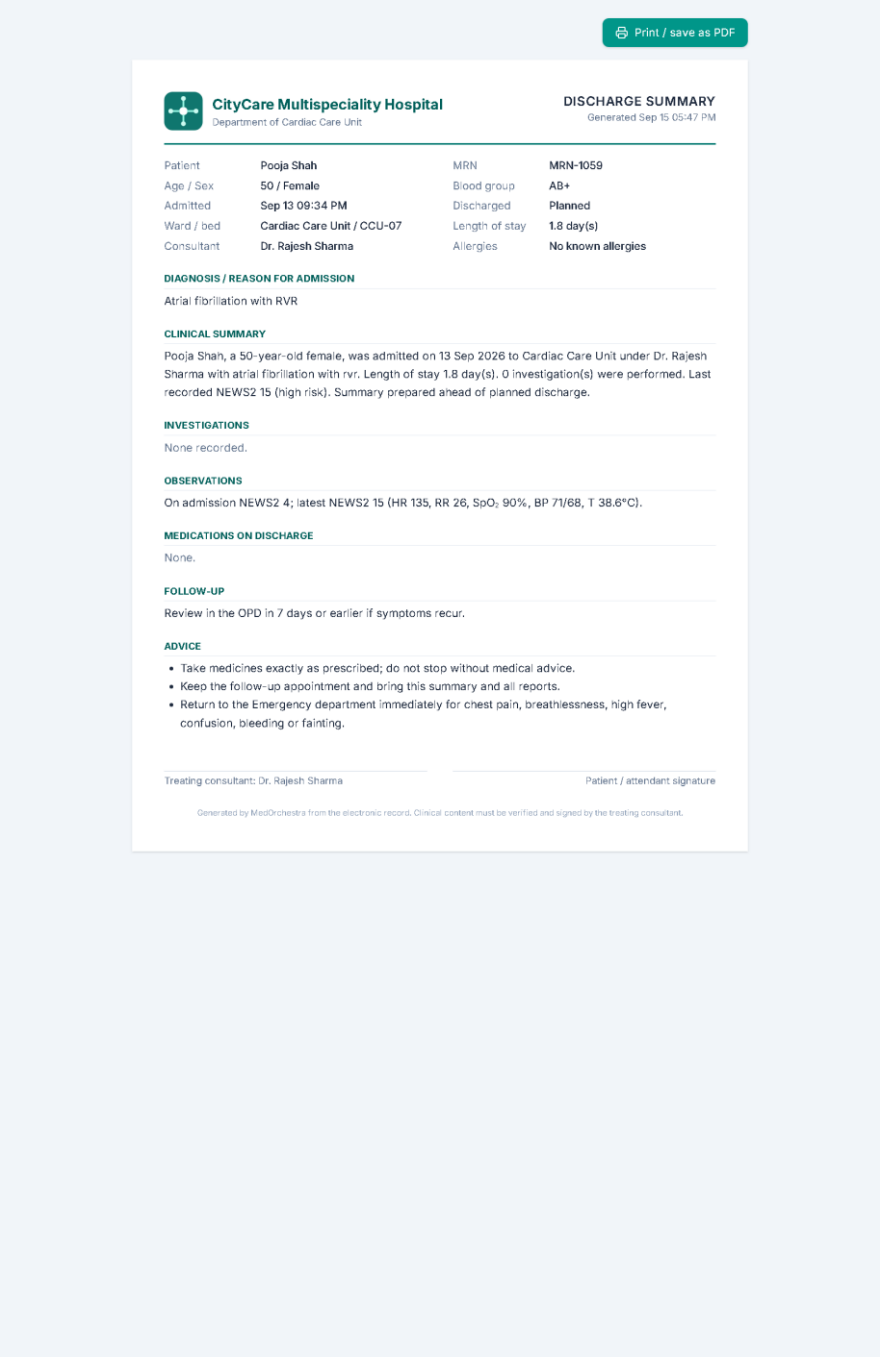


Figure 6.11: Printable discharge summary generated from the record

6.7 Resources

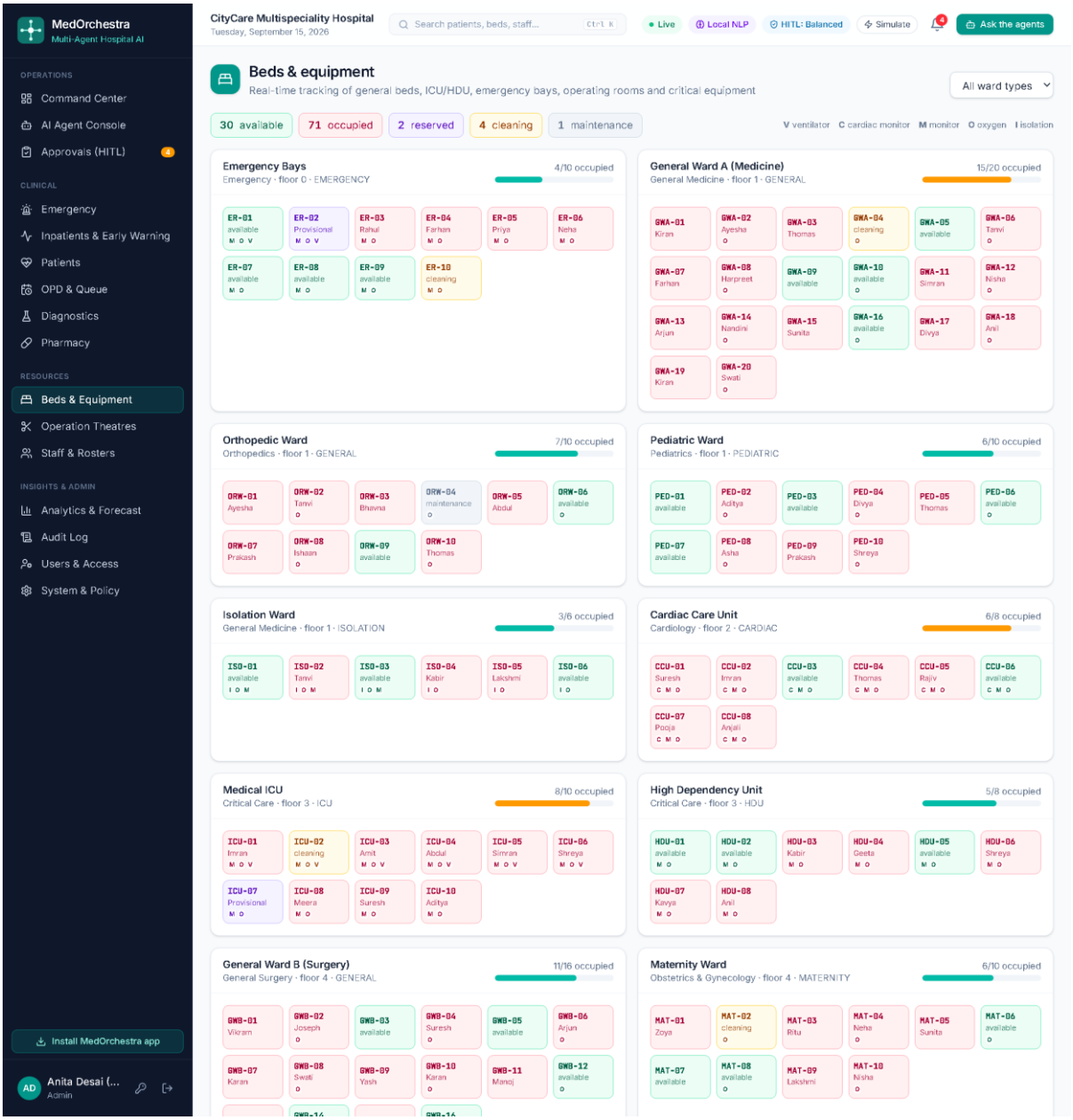


Figure 6.12: Beds and equipment with live status and predictive maintenance

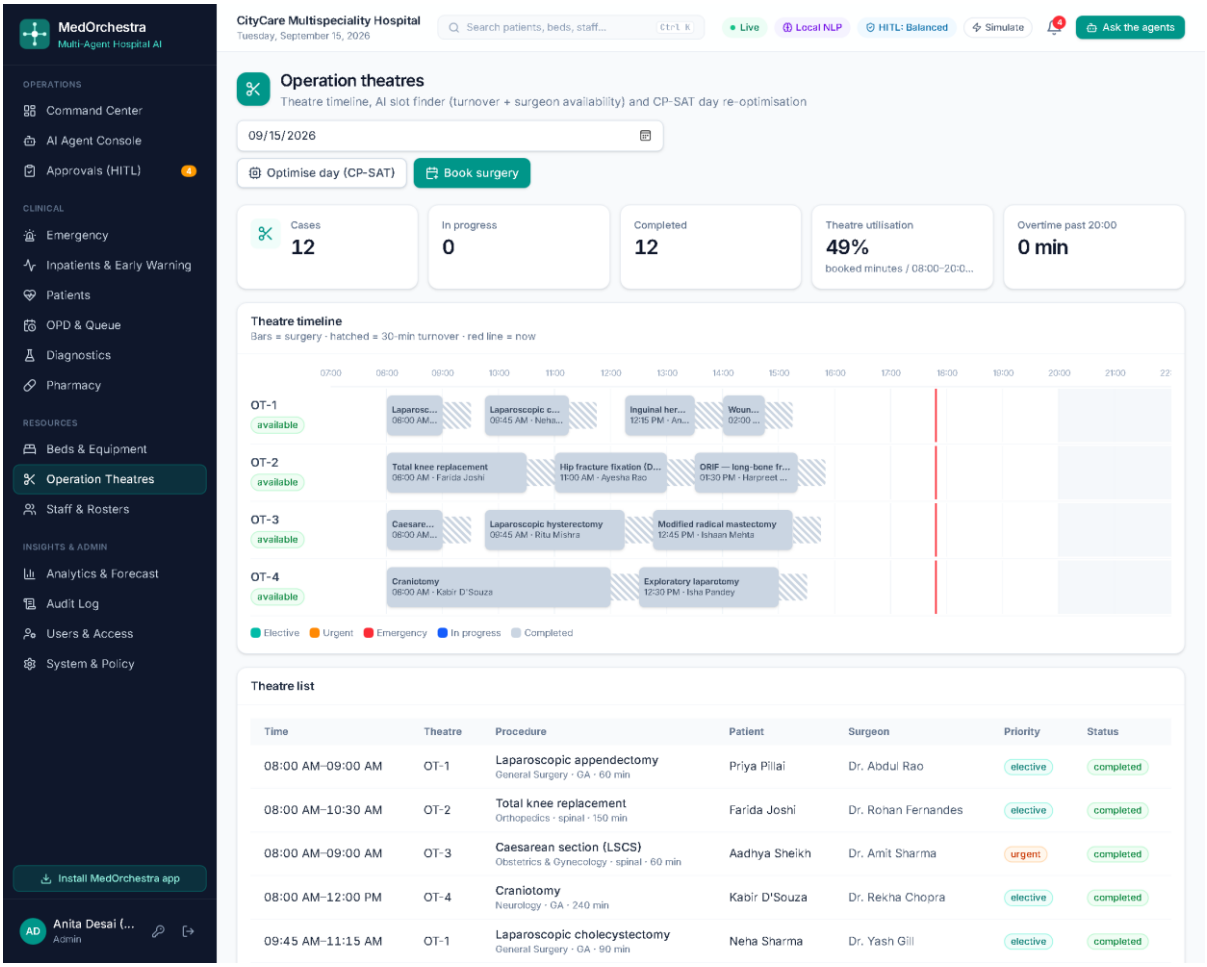


Figure 6.13: Operation-theatre timeline with turnover blocks and CP-SAT optimisation

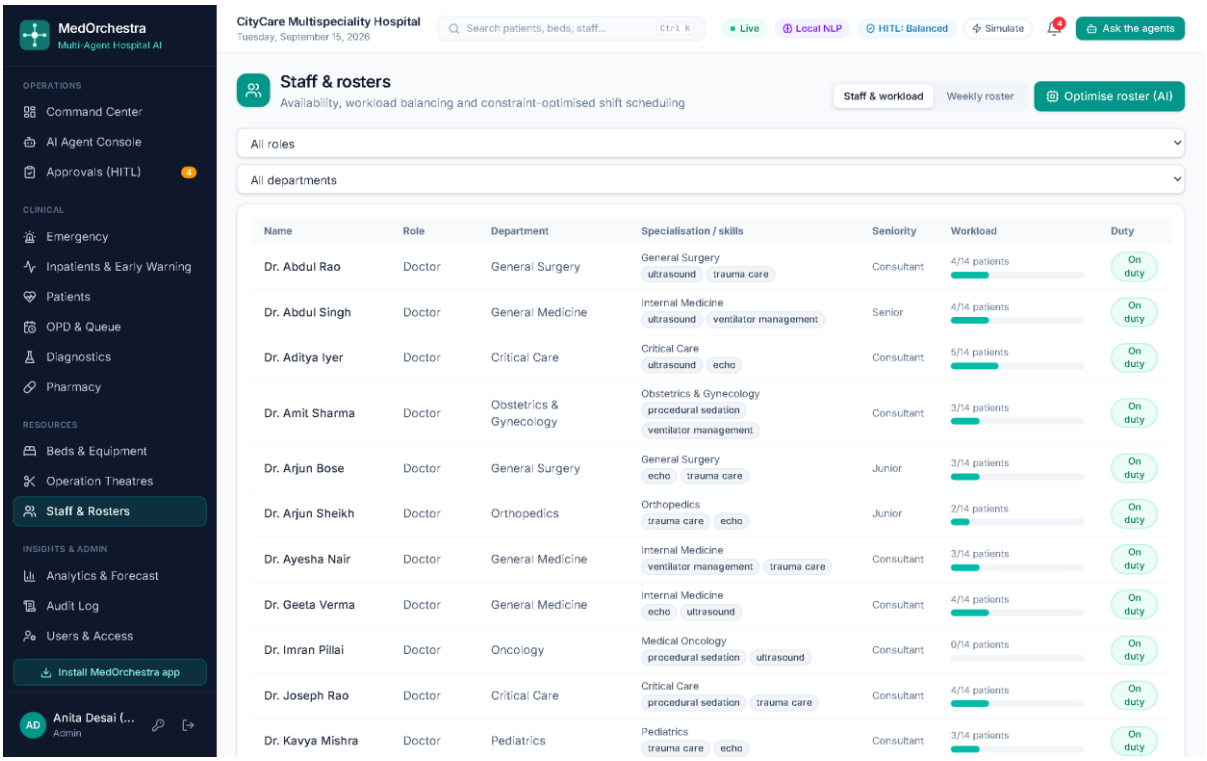


Figure 6.14: Staff workload and duty status

6.8 Clinical Workflows

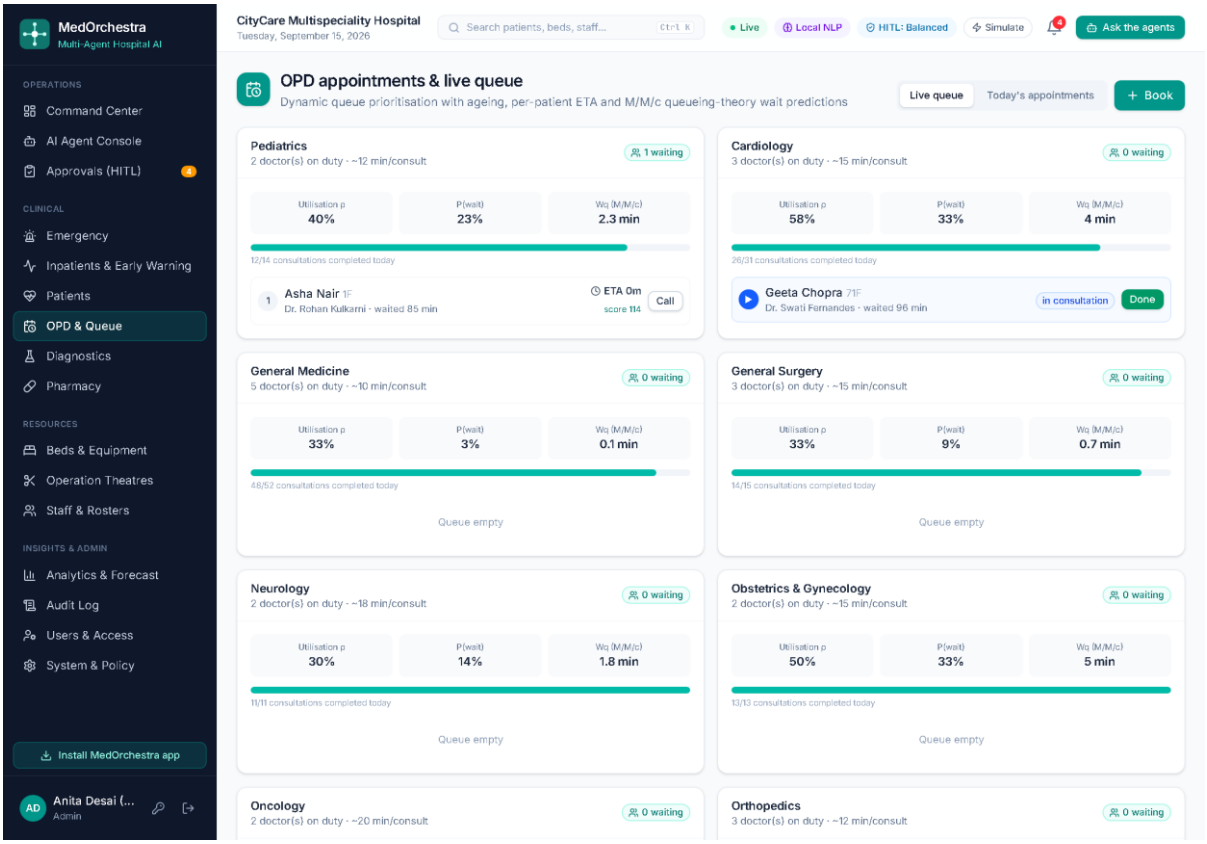


Figure 6.15: OPD queues with dynamic priority, ETA and M/M/c metrics

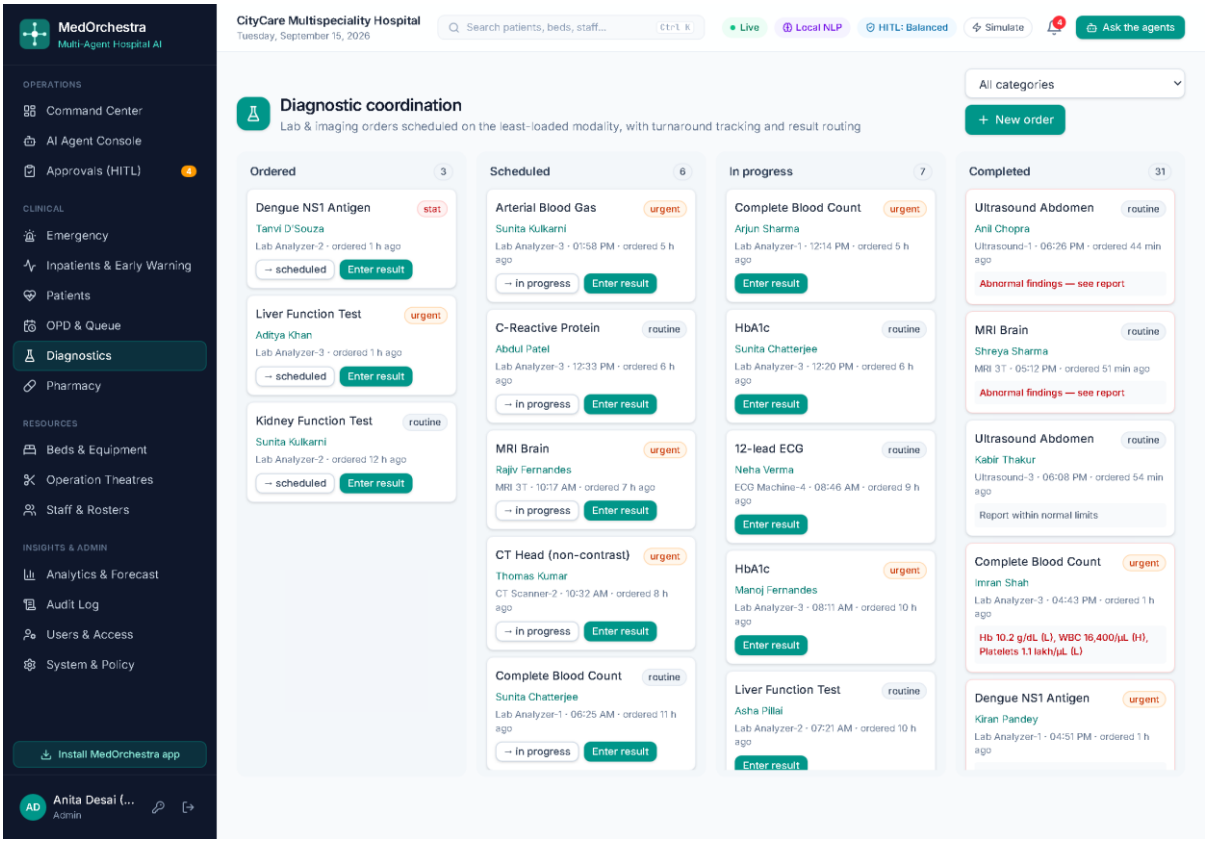


Figure 6.16: Diagnostics board with turnaround tracking and result reporting

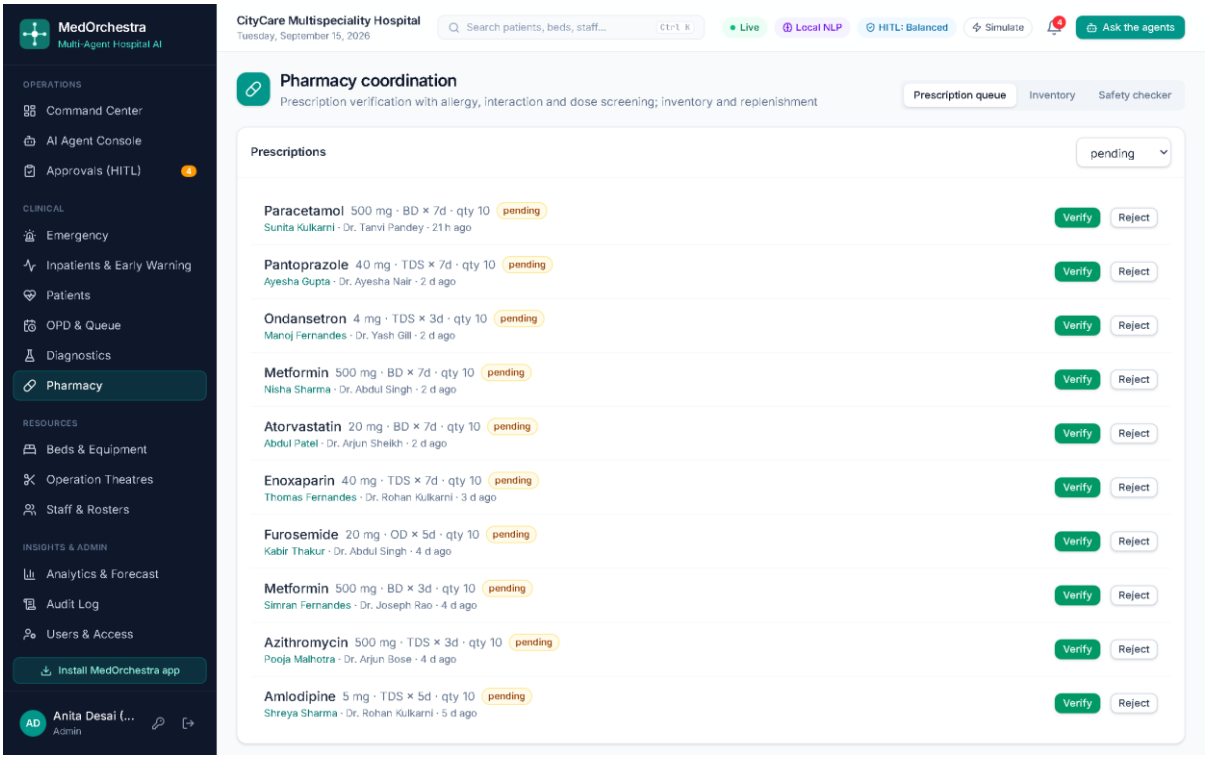


Figure 6.17: Pharmacy verification queue with safety alerts

6.9 Analytics, Administration and Patient Portal

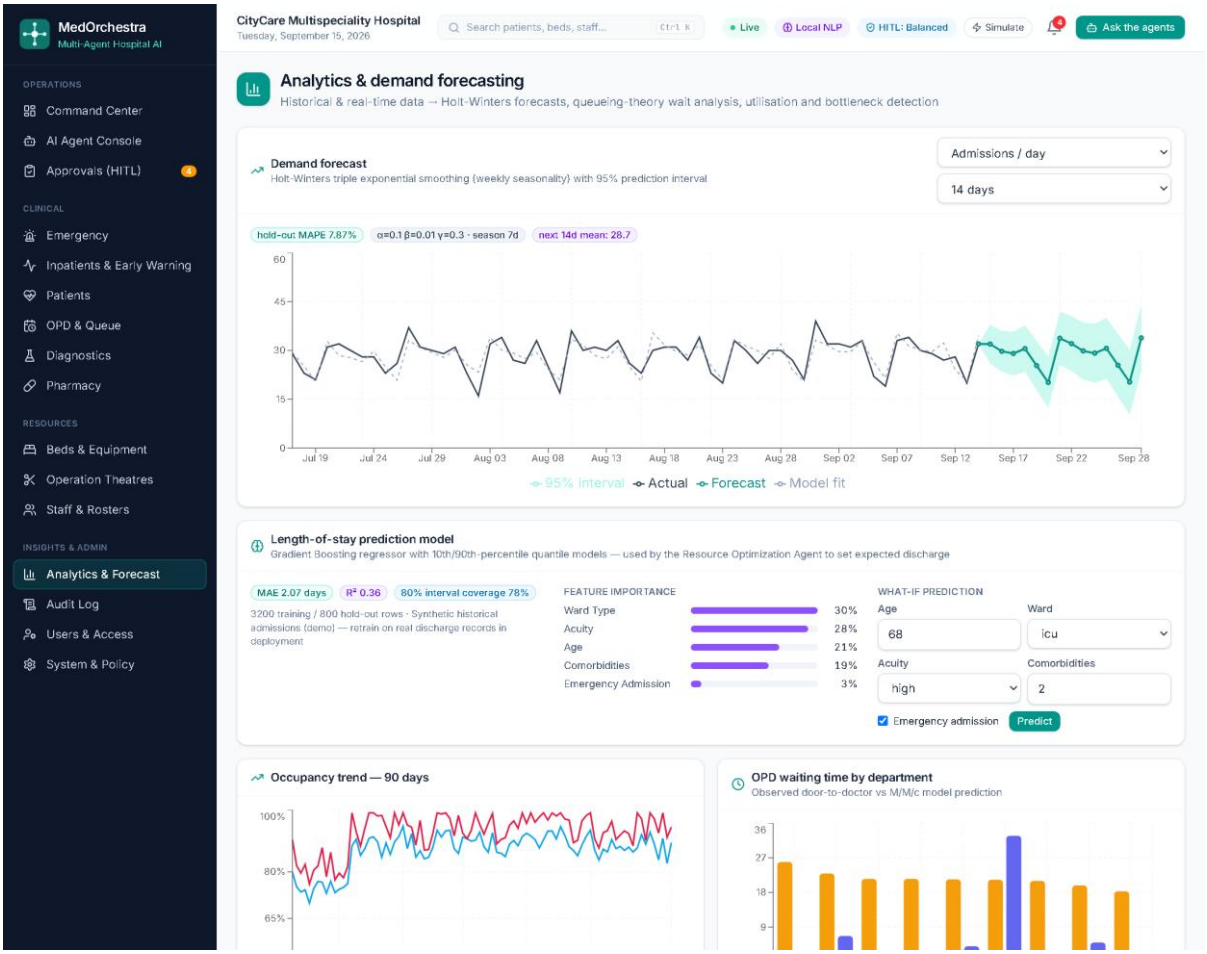


Figure 6.18: Holt-Winters demand forecast and length-of-stay model card

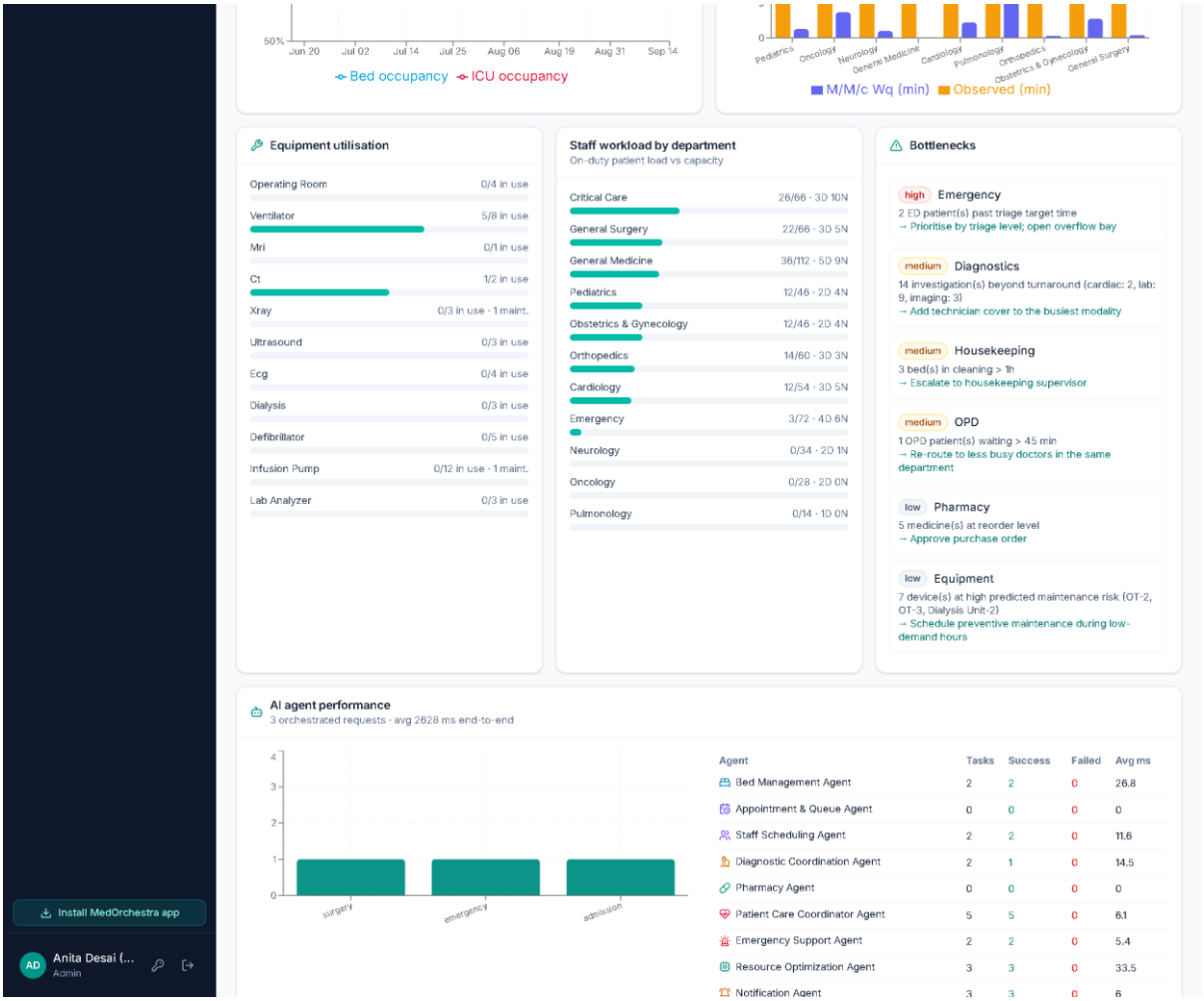


Figure 6.19: Occupancy, waiting-time, utilisation and agent-performance analytics

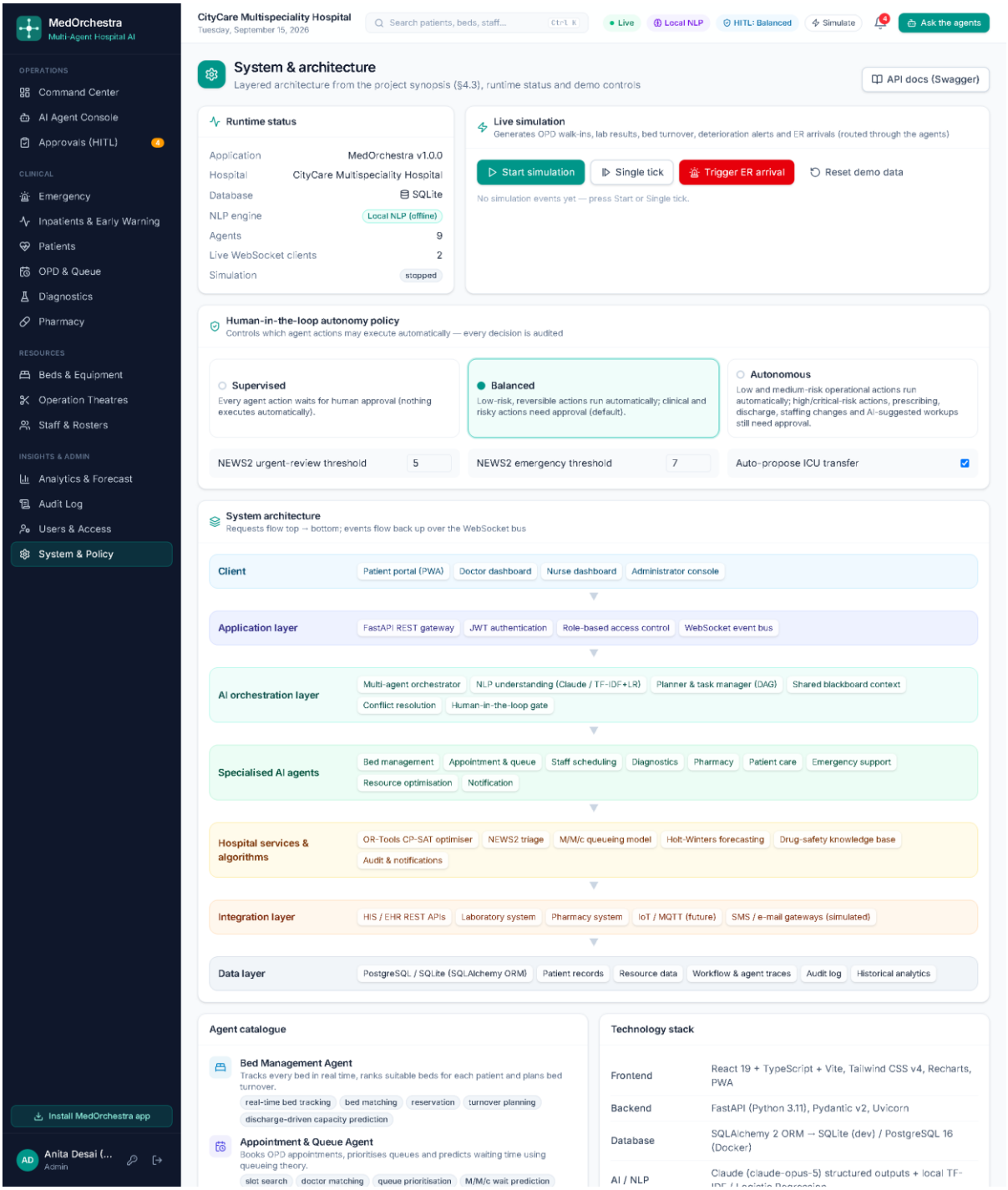


Figure 6.20: System & policy: runtime status, simulation, autonomy policy and architecture

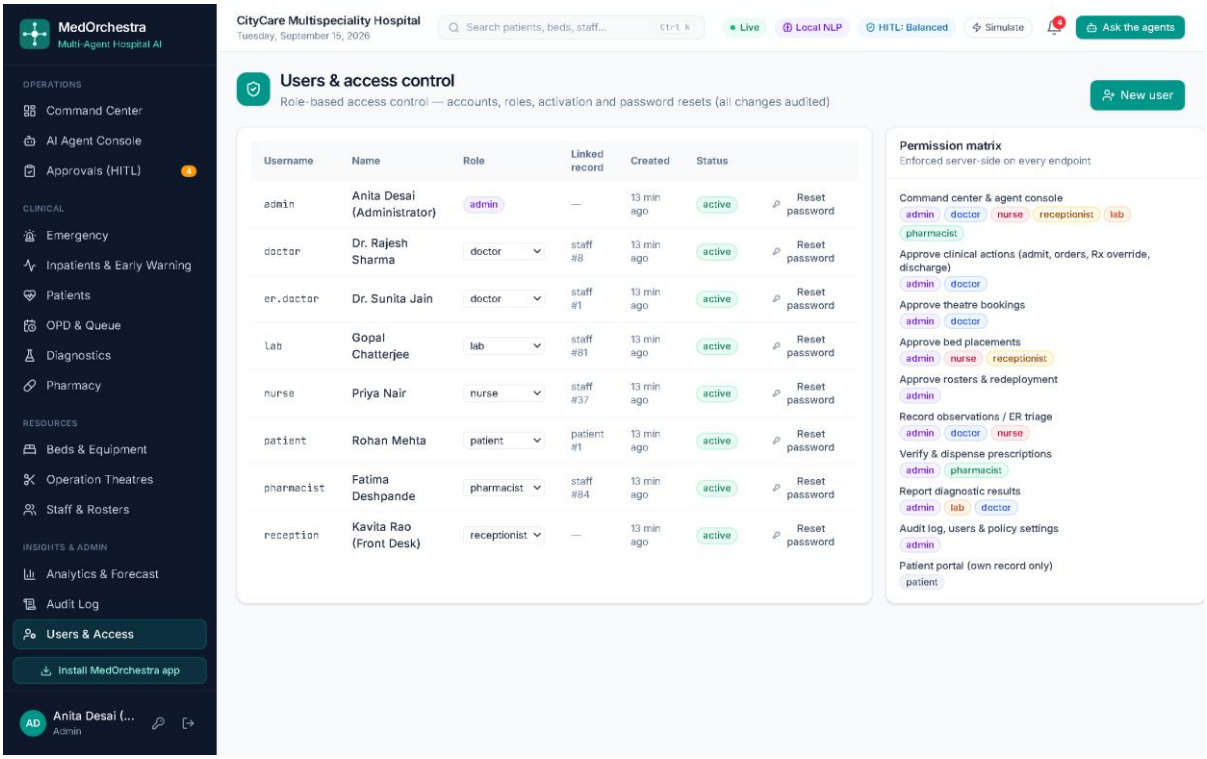


Figure 6.21: User and access management with the permission matrix

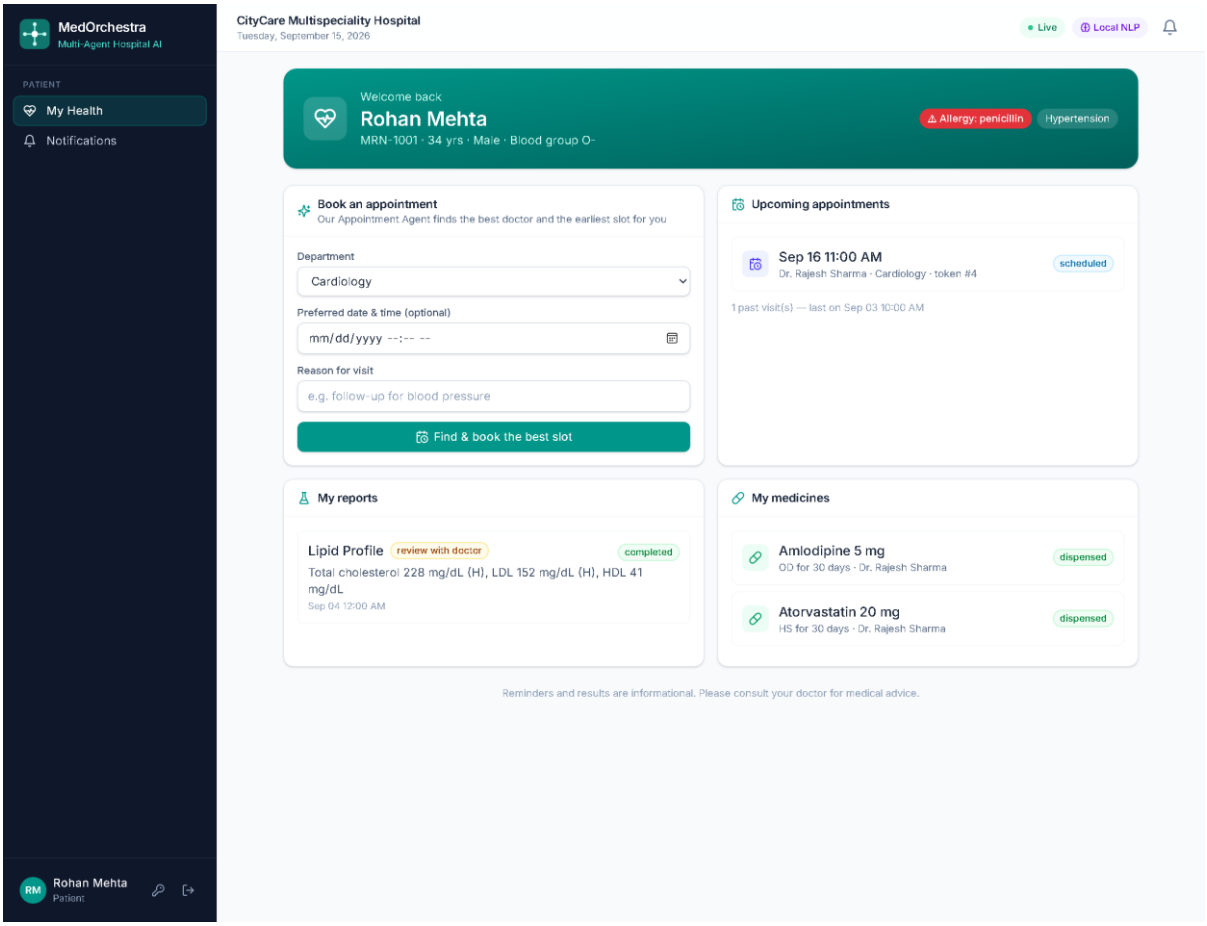


Figure 6.22: Patient portal with AI appointment booking

Chapter 7: Testing and Evaluation

7.1 Testing Strategy

- **Unit tests** for algorithms: NEWS2, triage, NLP, entity extraction, knowledge base, bed optimisation, rostering, queueing, forecasting and the LOS model.
- **Integration tests** through the REST API with a fresh database: authentication, RBAC, complete multi-agent orchestrations, approvals and rollbacks, ward escalation, theatre scheduling, autonomy policy and user management.
- **System tests** of end-to-end scenarios in the browser (Section 7.3), including live WebSocket updates.
- **Static checks**: the TypeScript compiler over the whole frontend and a production build.

7.2 Automated Test Results

The backend suite contains 38 pytest tests (19 unit, 19 integration). All 38 tests pass (`38 passed` in about 15 s). Table 7.1 groups them by area.

Table 7.1: Automated test cases

Area	What is verified	Result
Triage	NEWS2 is 0 for normal vitals; 14 for a severely abnormal set	Pass
Triage	Five-level triage for arrest, chest pain, sprain and unresponsive patients; age escalation	Pass
NLP	Single-intent classification (admission, appointment, bed query, analytics)	Pass
NLP	Multi-intent detection (admission + diagnostics); no false admission from “forecast admissions”	Pass
NLP	Entity extraction (age, sex, ICU, ventilator, BP, SpO_2, troponin, stat); °F→°C	Pass
Clinical KB	Longest test match; word-bounded protocol match; allergy and interaction detection	Pass
Optimiser	Hard constraints respected; premium equipment reserved; urgent patient placed first	Pass
Optimiser	Roster: ≤ 1 shift/day, ≤ 5 shifts/week, full coverage when feasible	Pass
Queueing / forecast	Erlang-C bounds and instability; priority ageing; weekly pattern captured (MAPE < 10 %)	Pass
Auth / RBAC	Login, wrong password, forbidden endpoints for nurse and patient, unauthenticated access	Pass
Orchestration	ICU admission → approval → admission executed	Pass
Orchestration	Chest-pain emergency → L1 triage; rejection releases the held bay	Pass
Safety	Penicillin-allergic patient: amoxicillin held as critical approval	Pass
Automation	Appointment booked automatically (low risk)	Pass
HITL	Nurse cannot approve an admin-only roster action	Pass

Area	What is verified	Result
Emergency API	Triage preview and ER registration	Pass
Portal	Patient sees own record only	Pass
ML	LOS model better than median baseline; interval coverage 70–90 %	Pass
Theatres	Empty day starts at 08:00; optimiser removes overtime without overlaps	Pass
Theatres	Surgery booked through the agents, no theatre clash	Pass
Early warning	NEWS2 ≥ 7 triggers emergency response and ICU-transfer proposal; normal vitals do not escalate	Pass
Policy	Supervised mode holds even an appointment; invalid mode rejected	Pass
Users	Create user, change password, deactivate account	Pass
Misc	Global search, role work lists, discharge summary, maintenance fields	Pass

7.3 Scenario Testing

Table 7.2: End-to-end scenario tests

Scenario (natural-language input)	Observed behaviour
“Patient needs admission and requires ICU monitoring with ventilator support.”	Provisional registration; ventilator ICU bed held; Critical Care doctor and ICU nurse selected; admission awaits doctor approval
“Emergency: 58 year old male with chest pain … BP 92/60, HR 124, SpO_2 93%”	NEWS2 8 → L1; resuscitation bay preferred; ED doctor + cardiology consult; ACS workup suggested for review; critical alerts
“Admit Rohan Mehta to the cardiac unit and order ECG, troponin and chest x-ray stat”	Two intents; requested tests ordered automatically; cardiac bed held; admission awaits approval
“Prescribe amoxicillin 500 mg BD for Rohan Mehta” (penicillin allergy)	Pharmacy Agent refuses; conflict logged; critical approval for doctor override
“Book an appointment with a cardiologist tomorrow at 11 am …”	Best doctor and slot found; booked automatically; SMS confirmation
“We need two more nurses in the ICU tonight”	Two lowest-workload nurses proposed for redeployment, keeping minimum cover
“Generate an optimized staff roster for next week”	CP-SAT roster for 77 staff, 86.5% coverage, awaiting admin approval
“Schedule a laparoscopic cholecystectomy for Rohan Mehta tomorrow”	Ranked theatre slots with explanations; surgeon selected; booking awaits approval; surgical pathway on execution
“Optimize the operation theatre schedule for tomorrow to reduce overtime”	Overtime 75 → 0 min, overlaps 1 → 0
Bedside observations with NEWS2 ≥ 7 on a general ward	Emergency alert to doctors and nurses; automatic ICU-transfer request prepared by the agents
Patient portal booking	Appointment Agent books the best slot and confirms within seconds

7.4 Evaluation of the AI Components

Intent classification

The local NLP pipeline was evaluated on 48 held-out requests (four per intent) written independently of the training sentences. Accuracy of the primary intent is **89.6%**, with an average processing time of 16.71 ms per request.

Table 7.3: Held-out intent accuracy by intent

Intent	Correct / total
admission	4/4
emergency	4/4
appointment	4/4
diagnostics	3/4
pharmacy	4/4
discharge	4/4
staffing	3/4
bed query	2/4
optimization	4/4
analytics	4/4
patient status	3/4
surgery	4/4

Demand forecasting

Holt-Winters forecasts were evaluated on the last 14 days of the 180-day history and compared with a seasonal-naive baseline (the value seven days earlier).

Table 7.4: Forecast accuracy (14-day hold-out)

Metric	Holt-Winters MAPE	Seasonal-naive MAPE
admissions	7.87%	11.43%
er visits	7.73%	7.77%
opd visits	3.68%	5.34%
bed occupancy	2.14%	3.36%
icu occupancy	3.16%	4.95%

Length-of-stay model

Table 7.5: Length-of-stay model evaluation

Measure	Value
Training / test rows	3200 / 800
MAE (Gradient Boosting)	2.07 days
MAE (median-stay baseline)	2.62 days
R^2	0.36
80 % interval coverage	77.7%
Most important features	ward type (30%), acuity (28%), age (21%), comorbidities (19%)

Length of stay is inherently noisy; the model reduces the error of the naive baseline by about one fifth and provides calibrated intervals, which is what planners need for discharge and capacity planning. Retraining on real discharge records would be required before clinical use.

Optimisation performance

Table 7.6: Optimisation results

Problem	Size	Result	Solve time
Bed allocation (CP-SAT)	8 patients × 32 free beds (68 feasible pairs)	8/8 placed, OPTIMAL	52.6 ms
Weekly roster (CP-SAT)	77 staff × 7 days × 3 shifts (1981 variables)	OPTIMAL; 86.5% of 392 slots; nights 0–2 per person	1.97 s
Theatre day (CP-SAT)	5 movable cases × 4 theatres	OPTIMAL; overtime 75→0 min; last case 20:45→11:15; overlaps 1→0	49.5 ms

Roster coverage below 100 % reflects genuinely infeasible requirements in small departments under the five-shift and rest constraints; the solver minimises the shortfall and reports it instead of silently violating constraints.

Orchestration latency

A complete ICU-admission orchestration (seven agent steps) finished in about 3.2 s, and a chest-pain emergency (nine steps) in about 3.5 s, including a deliberate 0.35 s visual pause per step that lets users follow the live trace; without the pause the same workflows complete in well under one second.

7.5 Discussion

The results show that a multi-agent design lets each operational sub-problem be solved with the most appropriate, verifiable technique while the orchestrator composes them into coherent

workflows. Explanations and the approval gate make the recommendations reviewable, and the audit log makes them accountable. The main limitation of the evaluation is the synthetic data: the absolute numbers demonstrate correctness and feasibility rather than clinical effectiveness, which would require a prospective study in a hospital.

Chapter 8: Conclusion and Future Scope

8.1 Conclusion

This project designed and implemented MedOrchestra, a multi-agent AI system for hospital resource optimisation, clinical workflow automation and patient-care coordination. Nine specialised agents, coordinated by a central orchestrator with a shared blackboard and explicit agent messages, convert natural-language requests into planned, explainable and auditable actions. Constraint optimisation allocates beds, rosters staff and schedules theatres; NEWS2 drives triage and ward escalation; queueing theory predicts waiting time; exponential smoothing forecasts demand; and a gradient-boosting model predicts length of stay. A risk-tiered, configurable human-in-the-loop gate keeps clinicians in control of every safety-critical decision. The working prototype covers all modules of the synopsis, extends them with inpatient monitoring, theatre scheduling and predictive maintenance, and passes 38 automated tests.

8.2 Limitations

- All data is synthetic; clinical effectiveness has not been evaluated in a real hospital.
- The clinical knowledge base (protocols, interactions, doses) is a demonstration subset and must be replaced by a validated drug and protocol database.
- Integrations with HIS/EHR, laboratory, pharmacy, SMS and IoT systems are simulated.
- The local NLP model is trained on template sentences; real requests would require more training data or an LLM.
- Real-world deployment must comply with healthcare data-protection and AI-governance regulations.

8.3 Future Scope

- **HL7 FHIR integration** with hospital information systems and laboratory instruments.
- **IoT and smart beds** streaming vital signs directly into the early-warning service.
- **Learning from real data**: retrain the LOS and demand models on hospital records and add readmission-risk prediction.
- **Multi-hospital coordination** for referrals and resource sharing across a network.
- **Voice and regional-language interfaces** for staff and patients.
- **Advanced explainability** (e.g. counterfactual “what would change this recommendation” explanations) and continuous monitoring of agent performance.
- **Mobile apps** with push notifications for on-call staff.

References

[1] M. Wooldridge, An Introduction to MultiAgent Systems, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.

[2] Foundation for Intelligent Physical Agents, "FIPA ACL Message Structure Specification," Document SC00061G, 2002.

[3] H. P. Nii, "Blackboard systems," AI Magazine, vol. 7, no. 2 and no. 3, 1986.

[4] Royal College of Physicians, National Early Warning Score (NEWS) 2: Standardising the Assessment of Acute-Illness Severity in the NHS. London: RCP, 2017.

[5] G. B. Smith, D. R. Prytherch, P. Meredith, P. E. Schmidt and P. I. Featherstone, "The ability of the National Early Warning Score (NEWS) to discriminate patients at risk of early cardiac arrest, unanticipated intensive care unit admission, and death," Resuscitation, vol. 84, no. 4, pp. 465–470, 2013.

[6] Manchester Triage Group, Emergency Triage, 3rd ed. Chichester, UK: Wiley-Blackwell, 2014.

[7] B. Cardoen, E. Demeulemeester and J. Beliën, "Operating room planning and scheduling: A literature review," European Journal of Operational Research, vol. 201, no. 3, pp. 921–932, 2010.

[8] L. V. Green, "Queueing analysis in healthcare," in Patient Flow: Reducing Delay in Healthcare Delivery, R. W. Hall, Ed. New York: Springer, 2006, pp. 281–307.

[9] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," Management Science, vol. 6, no. 3, pp. 324–342, 1960.

[10] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," International Journal of Forecasting, vol. 20, no. 1, pp. 5–10, 2004.

[11] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[12] K. Stone, R. Zwiggelaar, P. Jones and N. Mac Parthaláin, "A systematic review of the prediction of hospital length of stay: Towards a unified framework," PLOS Digital Health, vol. 1, no. 4, e0000017, 2022.

[13] L. Perron and V. Furnon, OR-Tools, Google. [Online]. Available: https://developers.google.com/optimization/

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[15] L. Kleinrock, Queueing Systems, Volume 1: Theory. New York: Wiley, 1975.

[16] World Health Organization, Ethics and Governance of Artificial Intelligence for Health: WHO Guidance. Geneva: WHO, 2021.

[17] S. Ramírez, FastAPI. [Online]. Available: https://fastapi.tiangolo.com

[18] Anthropic, Claude API documentation. [Online]. Available: https://docs.anthropic.com

[19] Meta Open Source, React. [Online]. Available: https://react.dev

[20] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, 2015.

[21] A. K. Erlang, “Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges,” Elektroteknikeren, vol. 13, 1917.

Appendix A: Installation and Execution

Prerequisites: Python 3.10+ and Node.js 18+. On Windows, run `setup.bat` once and then `start.bat`; the browser opens at http://localhost:5173.

```
cd backend
python -m venv .venv  &&  .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --port 8000

cd ../frontend
npm install
npm run dev                     # http://localhost:5173

docker compose up --build       # PostgreSQL + API + web on http://localhost:8080
cd backend && .venv\Scripts\python -m pytest -q    # 38 passed
```

The database is created and seeded automatically. Setting `ANTHROPIC_API_KEY` in `backend/.env` enables Claude-based understanding; without it the system runs fully offline. API documentation is available at http://localhost:8000/docs.

Appendix B: Demo Accounts and Walkthrough

Table 8.1: Demo accounts

Role	Username	Password
Administrator	admin	admin123
Doctor (Cardiology)	doctor	doctor123
ER doctor	er.doctor	doctor123
ICU nurse	nurse	nurse123
Reception	reception	reception123
Lab technician	lab	lab123
Pharmacist	pharmacist	pharma123
Patient	patient	patient123

11. Log in as admin; review the command center.
12. AI Agent Console → "ICU admission (synopsis example)"; follow the agent network and trace; open the explanations; approve the admission.
13. Emergency → "Chest pain" preset → register; observe triage and coordination; reject to see the bed released.
14. Console → "Prescription safety": the allergy is blocked for doctor override.
15. Inpatients → record observations with NEWS2 ≥ 7 to trigger an ICU-transfer proposal.
16. Operation Theatres → tomorrow → Optimise day (CP-SAT) → apply.
17. System & Policy → switch autonomy mode to Supervised and repeat an appointment booking.
18. Analytics → forecasts and the length-of-stay model; log in as patient to book through the portal.